

微机原理及接口技术

第4章 C51程序设计及应用

机械与汽车工程学院

目 录

- 4.1 C51简介
- 4.2 C51运算符及表达式
- 4.3 C51控制语句
- 4.4 数组
- 4.5 函数
- 4.6 指针
- 4.7 Keil 51单片机集成开发环境
- 4.8 Keil C 与Proteus联机调试示例

4.1 C51简介

C51是为51系列单片机进行编程设计的一种C语言，其特点：

- 结构化语言，**代码紧凑**——效率可与汇编语言媲美
- 接近真实语言，**程序可读性强**——易于调试、维护
- 库函数丰富，**编程工作量小**——产品开发周期短
- 机器级控制能力，**功能很强**——适合于嵌入式系统开发
- 与汇编指令无关，**易于掌握**——在单片机基础上上手快

C51语言已成为51系列单片机程序开发的主流软件方法。

C51与标准C程序结构完全相同

- ① 程序由函数组成（一个主函数，或一个主函数和若干自定义函数）；
- ② 利用预处理命令对变量或函数进行集中定义或说明；
- ③ 函数和变量都需遵循先定义后使用的基本原则；
- ④ 主函数中的所有语句执行完毕,则程序结束。

```
#include<头文件>
函数类型说明
全局变量定义
main(){
    局部变量定义
    <程序体>
}
func1(){
    局部变量定义
    <程序体>
}
:
funcN(){
    局部变量定义
    <程序体>
}
```

C51与标准C的主要区别

(1) **头文件的差异**。51系列单片机厂家有多个，它们的**差异在于内部资源如定时器、中断、I/O等数量以及功能的不同**，而对使用者来说，只需要将相应的功能寄存器的头文件加载在程序内，就可实现所具有的功能。因此，**Keil C51系列的头文件集中体现了各系列芯片的不同资源及功能**。

(2) **数据类型的不同**。51系列单片机包含**位操作**空间和丰富的位操作指令，因此Keil C51与ANSI C相比又**扩展了4种类型**，以便能够灵活地进行操作。电脑CPU是32位或64位，运算能力强，内存大，大量使用float型与long int型变量，单片机一般为8位或16位，运算能力较弱，以char型为主,int型为辅；

C51与标准C的主要区别

- (3) **数据存储类型的不同**。C语言最初是为通用计算机设计的，在通用计算机中只有一个程序和数据统一寻址的内存空间，而**51系列单片机**有**片内、外程序存储器**，还有**片内、外数据存储器**。标准C并没有提供这部分存储器的地址范围的定义，此外，对于**51单片机**中大量的**特殊功能寄存器**也没有定义。
- (4) 标准C语言没有处理单片机中断的定义。
- (5) Keil C51与标准C的**库函数**有较大的不同。

由于**标准C的部分库函数不适于嵌入式处理器系统**，因此被排除在Keil C51之外，如字符屏幕和图形函数。

C51与标准C的主要区别

(6) 目标代码

电脑生成.exe格式，编译完成后直接在电脑上运行，单片机编译生成.bin代码文件，需要烧写到单片机中并结合外围电路执行。

(7) 仿真调试

单片机的C程序在电脑上进行编译，然后需通过仿真器连接后进行仿真调试，近年来随着电路仿真软件的不断完善，也可通过proteus等软件直接在电脑上进行仿真调试。

4.1.1 C语言的标识符和关键字

标识符是用来标识源程序中某个对象的名字的，这些对象可以是语句、数据类型、函数、变量、常量、数组等。一个标识符有字符串、数字和下划线等组成，第一个字符必须是字母或者下划线，C编译程序识别大小写英文字母。为便于阅读和理解程序，标识符应该以含义清晰的字符组合命名。

关键字则是编程语言保留的特殊标识符，有时又称为保留字，它们具有固定名称和含义。C语言关键字见表4-1。



ANSI C标准规定的关键字

关键字	用途	说明
auto	存储种类说明	用以说明局部变量，缺省值为此
break	程序语句	退出最内层循环
case	程序语句	Switch语句中的选择项
char	数据类型说明	单字节整型数或字符型数据
const	存储种类说明	在程序执行过程中不可更改的常量值
continue	程序语句	转向下一次循环
default	程序语句	Switch语句中的失败选择项
do	程序语句	构成 do…while循环结构
double	数据类型说明	双精度浮点数
else	程序语句	构成 if…else选择结构
enum	数据类型说明	枚举类型
extern	存储种类说明	在其他程序模块中说明了的全局变量
float	数据类型说明	单精度浮点数
for	程序语句	构成for循环结构
goto	程序语句	构成goto转移结构



if	程序语句	构成 if…else选择结构
int	数据类型说明	基本整型数
long	数据类型说明	长整型数
register	存储种类说明	使用CPU内部寄存器的变量
return	程序语句	函数返回
short	数据类型说明	短整型数
signed	数据类型说明	有符号数，二进制数据的最高位为符号位
sizeof	运算符	计算表达式或数据类型的字节数
static	存储种类说明	静态变量
struct	数据类型说明	结构类型数据
switch	程序语句	构成Switch选择结构
typedef	数据类型说明	重新进行数据类型定义
union	数据类型说明	联合类型数据
unsigned	数据类型说明	无符号数数据
void	数据类型说明	无类型数据
volatile	数据类型说明	该变量在程序执行中可被隐含地改变
while	程序语句	构成while和do…while循环结构

4.1.2 C51的扩展

Cx51编译器兼容ANSI C标准，又扩展支持了8051微处理器，其扩展内容如下：

- ① 存储区；
- ② 存储区类型；
- ③ 存储模型；
- ④ 存储类型说明符；
- ⑤ 变量数据类型说明符；
- ⑥ 位变量和位可寻址数据；
- ⑦ SFR；
- ⑧ 指针；
- ⑨ 函数属性。



表 4-2 · C51 增加的关键字

<code>_at_</code>	为变量定义存储空间绝对地址
<code>alien</code>	声明与 PL/M51 兼容的函数
<code>bdata</code>	可位寻址的内部 RAM
<code>bit</code>	位类型
<code>code</code>	ROM
<code>compact</code>	使用外部分页 RAM 的存储模式
<code>data</code>	直接寻址的内部 RAM
<code>idata</code>	间接寻址的内部 RAM
<code>interrupt</code>	中断服务函数
<code>large</code>	使用外部 RAM 的存储模式
<code>pdata</code>	分页寻址的外部 RAM
<code>_priority_</code>	RTX51 的任务优先级
<code>reentrant</code>	可重入函数
<code>sbit</code>	声明可位寻址的特殊功能位
<code>sfr</code>	8 位的特殊功能寄存器
<code>sfr16</code>	16 位的特殊功能寄存器
<code>small</code>	内部 RAM 的存储模式
<code>_task_</code>	实时任务函数
<code>using</code>	选择工作寄存器组
<code>xdata</code>	外部 RAM

C51增加右表所列关键字
对51单片机（微处理器）进
行支持

4.1.3 存储区、存储类型及存储模式

1. 存储区和存储类型

51单片机**程序存储器**和**数据存储器**分别独立编址。

存储器根据读写情况可以分为程序存储区（ROM）、快速读写存储器（内部RAM）及随机读写存储器（外部RAM）。

(1) 程序存储器(**code**)

在8051中程序存储器是**只读存储器**，其空间为64KB，在C51中用**code**关键字来声明访问程序存储区中的变量。

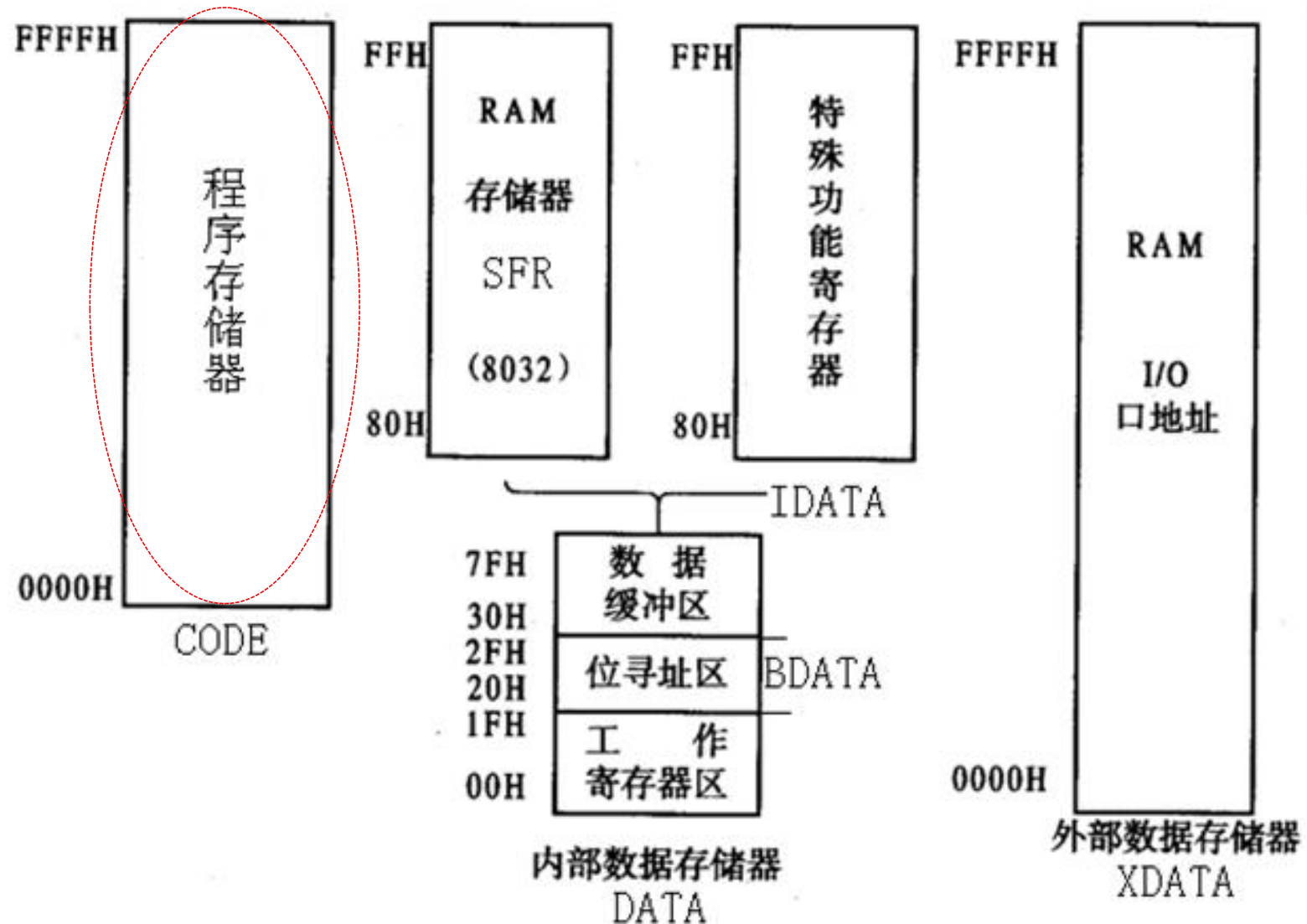


图4-1· C51存储类型与51单片机存储空间

1. 存储区和存储类型

(2) 内部数据存储器

在8051单片机中，内部数据存储器属于快速可读写存储器，与51兼容的扩展型单片机最多有256字节内部数据存储区。

1) data

data存储类型声明的变量可以对内部RAM直接寻址128B (0x00 ~ 0x7F)。

2) idata

idata存储类型声明的变量可以对内部RAM间接寻址256B (0x00 ~ 0xFF)，访问速度比data略慢

3) bdata

bdata存储类型声明的变量可以对内部RAM 16B (0x20 ~ 0x2F) 的128位进行位寻址

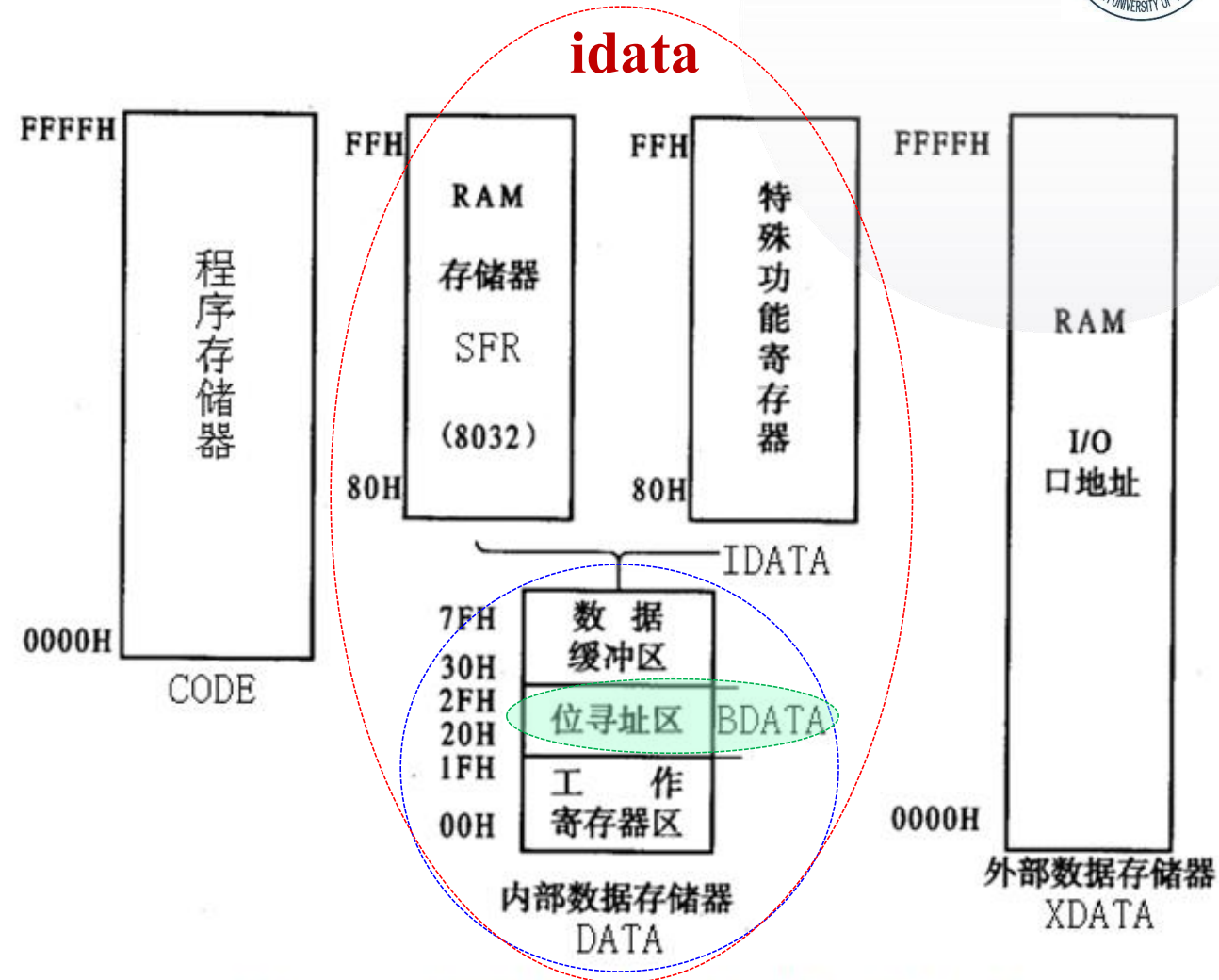


图4-1· C51存储类型与51单片机存储空间

1. 存储区和存储类型

(3) 外部数据存储器

外部数据存储器又称**随机读写存储器**，访问存储空间为**64KB**。其访问速度要比内部RAM慢。访问外部RAM的数据要使用指针进行**间接访问**。

在C51中使用关键字**xdata**和**pdata**存储类型声明的变量来访问外部存储空间中的数据。

1) xdata

xdata存储类型声明的变量可以**访问外部存储器64KB**的任何单元（0x0000 ~ 0xFFFF）。

2) pdata

pdata存储类型声明的变量可以访问外部存储器（一页）**低256B**（类似@R0,R1访问）。

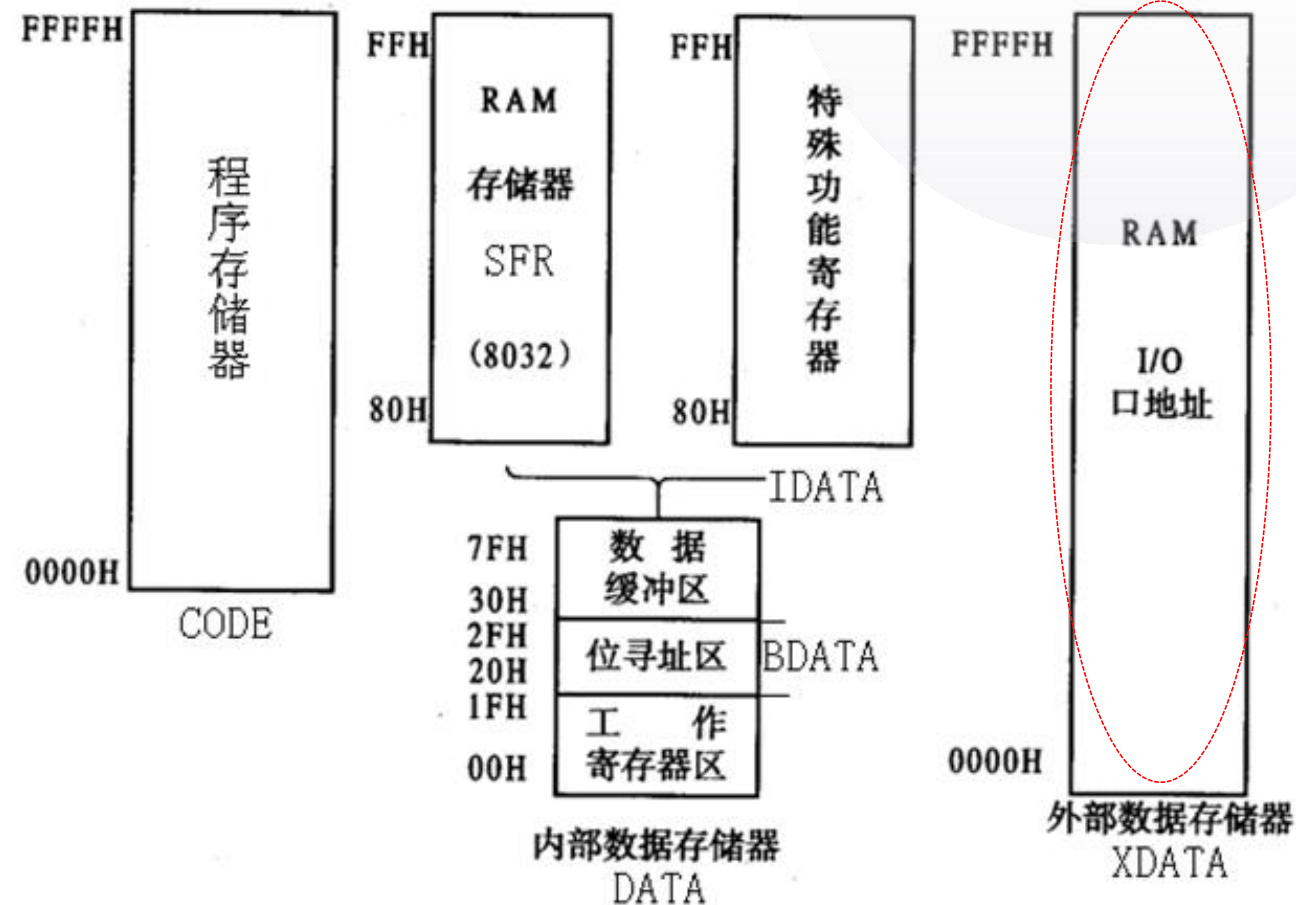


图4-1· C51存储类型与51单片机存储空间

51单片机存储类型及存储区

类型 关键字	存储区	描述
data	DATA	单片机内部RAM空间的低128字节（0x00-0x7f），可在一个周期内直接寻址。
bdata	BDATA	DATA区中可以字节、位混合寻址的16字节（0x20-0x2f）位寻址区（0x00-0x7f）。
idata	IDATA	内部RAM 256字节（0x00-0x0ff），采用间接寻址。
xdata	XDATA	片外RAM，地址范围64KB。
pdata	PDATA	片外RAM的256字节(如 0x0000-0x00ff, 0x0ff00~0x0ffff)。
code	CODE	程序存储区64KB (0x0000-0x0ffff)，内容只读。

访问频繁的小变量采用**data**，片内RAM直接寻址，执行效率高，空间有限情况下也可以用idata(间接寻址,略慢)；而占用内存空间大但访问不频繁的使用**xdata**，例如：

```
data unsigned char c1, c2; //访问频繁，占用内存少，
```

```
xdata unsigned dat_tab[100]; // 数组，使用内存多，访问不频繁，片外RAM空间
```

2. 存储模式

在C51中，存储器模式可以确定一些变量在缺省情况下的存储器类型。程序中可用**编译器控制命令SMALL、COMPACT、LARGE指定存储器模式**。

在**SMALL模式**中，程序中所有的变量位于单片机的内部RAM数据区，这和用data存储类型标识符声明的变量是相同的。由于**SMALL模式中的变量访问速度最快且效率高**，所以对于经常使用的变量应置于内部RAM中。

SMALL模式是C51编译器在缺省的情况下**默认的存储器类型**，一般情况下应使用SMALL存储模式。

2. 存储模式：

Small模式，所有缺省变量参数均装入**内部RAM**。

优点是速度快， 缺点在于空间有限， 仅适用于小规模程序设计。

Compact模式，所有缺省变量均位于**外部RAM区的一页**

(256个字节)， 具体哪一页可由P2口指定（在STARTUP.A51文件中说明， 也可用pdata指定）。 优点是可用空间较Small宽裕， 速度比Small慢但比Large要快。

Large模式，所有缺省变量可放在多达**64KB的外部RAM区**。

优点在于空间大， 可存变量多， 缺点是速度较前两种模式要慢。

一般使用**SMALL**模式，对部分容量大的变量，手动指定存储器类型（如定义为**xdata**类型）。

4.1.4 数据类型及变量

在Cx51中不仅支持所有的C语言标准数据类型，而且还对其进行了扩展，增加了专用于访问8051硬件的数据类型，使其对单片机的操作更加灵活。

在C51中，允许用户通过位类型符定义位变量。位类型符有两个：`bit`和`sbit`。可以定义两种位变量。

1. `bit`类型

`bit`位类型符用于声明**位变量**，其值为1或0。

定义格式：`bit 位变量名;`

在格式中也可以指定存储器类型，如**`bdata`**、**`data`**、**`idata`**。即使不指定存储器类型，对于用**`bit`**声明的变量，编译器会自动分配到**片内RAM的可位寻址区**。

```
bit bdata bflag0;    //位变量bflag0, 存储于片内位寻址区。
```

```
bit KeyPress;        // 位变量KeyPress, 将自动分配存储于片内位寻址区
```

```
bit data a1;    /*正确*/
bit bdata a2;   /*正确*/
bit idata a3;   /*正确*/
```

```
bit pdata a3;   /*错误*/
bit xdata a4;   /*错误*/
```

**** 位变量不能声明为指针类型或数组**

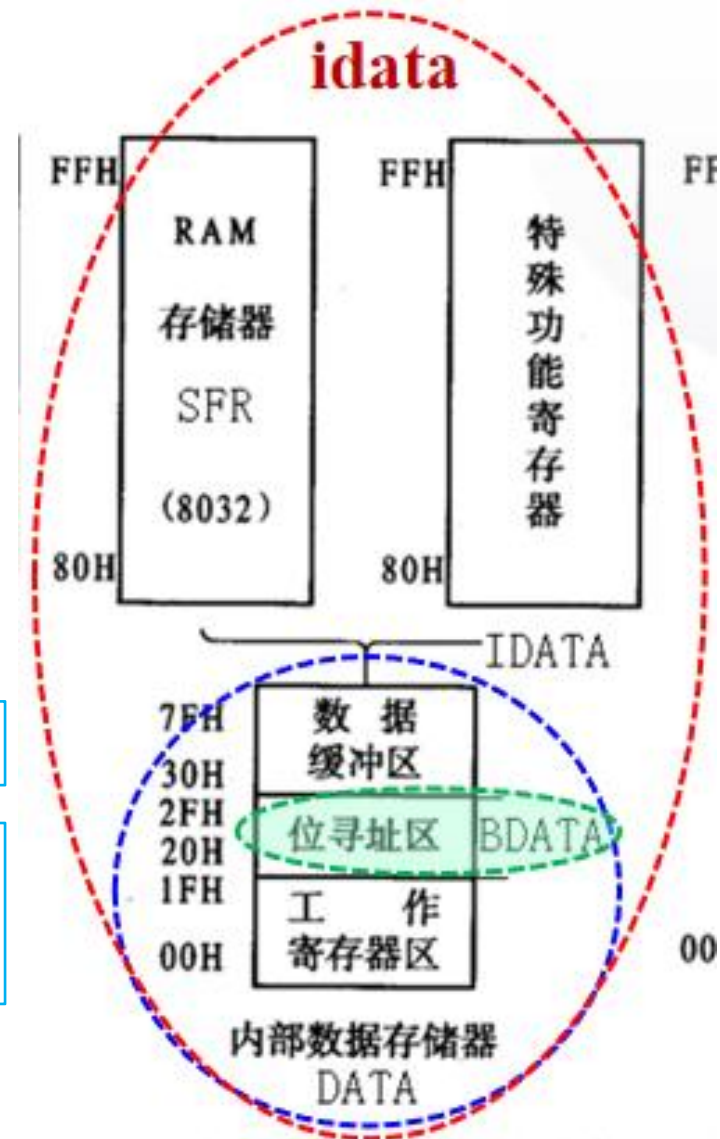
2. sbit类型

sbit类型用于声明**可位寻址变量或特殊功能寄存器(SFR)中的某个位变量**，其值为1或0。

➤ 可位寻址的字节变量地址: 0x20~0x2F

➤ 可位寻址的特殊功能寄存器(SFR):
ACC, B, PSW, P0~P3, IP, IE, TCON, SCON

* 可位寻址的寄存器，其字节地址的低4位必定是0H或者是8H



定义格式: **sbit 位变量名=位地址;**

如位地址是**可位寻址变量带位号**或**特殊功能寄存器名带位号**, 则须**先对可位寻址变量或特殊功能寄存器进行定义**。字节地址与位号之间、特殊功能寄存器与位号之间一般用 “^” 作间隔。

sbit用于定义位地址变量的三种定义形式:

- 1) 将 **绝对位地址** 定义为 位变量名 // 头文件.h 常用

sbit bit_name = 位地址常数;

例如, **sbit** CY = 0xD7;

- 2) 以**SFR变量名**的**相对位位置**定义位变量名 //常用

sbit bit_name = sfr_name ^ 位位置;

例如, **sbit** CY = PSW^7;

```
/* BIT Register */
/* PSW */
sbit CY = 0xD7;
sbit AC = 0xD6;
sbit F0 = 0xD5;
sbit RS1 = 0xD4;
sbit RS0 = 0xD3;
sbit OV = 0xD2;
sbit P = 0xD0;
```

3) 将可位寻址的**字节地址的相对位地址**定义为位变量名

sbit bit_name = **sfr**字节地址 ^ 位位置;

例如, **sbit** CY = 0xD0^7; //PSW寄存器的(字节地址0xD0)的第7位

sbit bit_name = 可位寻址**RAM** ^ 位位置; // **常用**, 可位寻址**RAM** 0x20~0x2f

例如, **sbit** sgnb0 = 0x20^1; // 定义可位寻址字节(0x20)的第1位地址

【例】sbit型变量定义:

```
sbit OV=0xd2; //位直接地址, SFR的位
```

```
sbit CY=0xd7; //位直接地址
```

```
unsigned char bdata flag; //定义可位寻址字  
节变量
```

```
sbit flag0=flag^0; //可位寻址变量带位号
```

```
sfr P1=0x90; //先定义特殊功能寄存器  
(SFR)
```

```
sbit P1_0=P1^0; //SFR带位号
```

```
sbit P1_1=P1^1;
```

```
sbit P1_2=P1^2;
```

* 位直接地址, 也称为**绝对位地址**

3.bdata类型及变量

bdata用于声明可位寻址的字节变量（8位）。对于用bdata声明的变量，编译器会自动分配到片内RAM的可位寻址区。

➤ 可位寻址的字节变量地址: 0x20~0x2F

```
unsigned char bdata flag; //定义可位寻址字节变量  
sbit flag0=flag^0; //可位寻址变量带位号
```


4. 特殊功能寄存器变量(sfr)

特殊功能寄存器(SFR)用于控制定时器、计数器、串口、I/O及其它功能部件，每一个SFR在片内RAM中对应一个字节单元或两个字节单元(位于片内RAM 0x80~0x0ff)。

通过sfr类型说明符定义这些特殊功能寄存器，定义时须指明它们所对应的片内RAM单元的地址。格式如下：

sfr 特殊功能寄存器名=地址;

sfr用于定义单字节的特殊功能寄存器。特殊功能寄存器名一般用大写字母表示，地址一般用直接地址形式。

【例】特殊功能寄存器定义

```
sfr PSW=0xd0;    //定义PSW地址
sfr SCON=0x98;
sfr P1=0x90;      //定义P1口地址
sfr DPL=0x82;    //定义DPTR的低字节地址82H
sfr DPH=0x83;    //定义DPTR的高字节地址83H
```

```
/* BYTE Register */
sfr P0    = 0x80;
sfr P1    = 0x90;
sfr P2    = 0xA0;
sfr P3    = 0xB0;
sfr PSW   = 0xD0;
sfr ACC   = 0xE0;
sfr B     = 0xF0;
sfr SP    = 0x81;
sfr DPL   = 0x82;
sfr DPH   = 0x83;
```


5. sfr16

用于声明两个连续地址的特殊功能寄存器(地址范围, 片内RAM 0x80~0x0ff, 特殊功能寄存器区)

如8052的定时器/计数器2的声明

sfr16 T2=0xCC; //T2为16位SFR, 低字节0xCC, 高字节0xCD

一般常用sfr 定义:

```
/* 8052 Extensions */  
sfr T2CON = 0xC8;  
sfr RCAP2L = 0xCA;  
sfr RCAP2H = 0xCB;  
sfr TL2 = 0xCC;  
sfr TH2 = 0xCD;
```

```
sfr DPL = 0x82;  
sfr DPH = 0x83;  
sfr PCON = 0x87;  
sfr TCON = 0x88;  
sfr TMOD = 0x89;  
sfr TL0 = 0x8A;  
sfr TL1 = 0x8B;  
sfr TH0 = 0x8C;  
sfr TH1 = 0x8D;
```

keil等编译器自带的头文件(“reg51.h”或“reg52.h”)已经定义了(51)单片机的全部特殊功能寄存器和特殊位,通过#include <reg51.h>(根据选用的单片机添加)把这个头文件包含到C51程序中,即可使用所有特殊功能寄存器和特殊位名称,无需重新定义。

```
main.c  reg51.h
1  /*-----
2  REG51.H
3
4  Header file for generic 80C5
5  Copyright (c) 1988-2002 Keil
6  All rights reserved.
7  -----
8
9  #ifndef __REG51_H__
10 #define __REG51_H__
11
12 /* BYTE Register */
13 sfr P0 = 0x80;
14 sfr P1 = 0x90;
15 sfr P2 = 0xA0;
16 sfr P3 = 0xB0;
17 sfr PSW = 0xD0;
18 sfr ACC = 0xE0;
19 sfr B = 0xF0;
20 sfr SP = 0x81;
21 sfr DPL = 0x82;
22 sfr DPH = 0x83;
23 sfr PCON = 0x87;
24 sfr TCON = 0x88;
```

```
25 sfr TMOD = 0x89;
26 sfr TLO = 0x8A;
27 sfr TL1 = 0x8B;
28 sfr TH0 = 0x8C;
29 sfr TH1 = 0x8D;
30 sfr IE = 0xA8;
31 sfr IP = 0xB8;
32 sfr SCON = 0x98;
33 sfr SBUF = 0x99;
34
35
36 /* BIT Register */
37 /* PSW */
38 sbit CY = 0xD7;
39 sbit AC = 0xD6;
40 sbit FO = 0xD5;
41 sbit RS1 = 0xD4;
42 sbit RS0 = 0xD3;
43 sbit OV = 0xD2;
44 sbit P = 0xD0;
45
46 /* TCON */
47 sbit TF1 = 0x8F;
48 sbit TR1 = 0x8E;
49 sbit TF0 = 0x8D;
```

```
70
71 /* P3 */
72 sbit RD = 0xB7;
73 sbit WR = 0xB6;
74 sbit T1 = 0xB5;
75 sbit T0 = 0xB4;
76 sbit INT1 = 0xB3;
77 sbit INT0 = 0xB2;
78 sbit TXD = 0xB1;
79 sbit RXD = 0xB0;
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2
```

应用举例:

```
→ #include <REG51.H>           //51 单片机头文件↵
void delay();                   //延时函数声明↵
→ sbit p1_0=P1^0;              //输出端口定义↵
main() {                        //主函数↵
    while(1) {                 //无限循环体↵
        → p1_0=0;              //P1.0="0",led 亮↵
        delay();               //延时↵
        → p1_0=1;              //P1.0="1",led 灭↵
        delay();               //延时↵
    }↵
}↵
↵
void delay (void) {             //延时函数↵
    unsigned char i;           //字符型变量 i 定义↵
    for (i=200;i>0;i--);       //循环延时↵
}↵
```

6. 其它类型

C51程序中常用的数据类型还有:

char (字符型)

声明一个字节的字符变量

unsigned char (无符号字符型)

int (整型)

unsigned int (无符号整型) 等类型。



表 5-1 Keil C51 支持的数据类型

数据类型	位数	字节	数值范围
signed char	8	1	-128 到+127
unsigned char	8	1	0 到 255
signed short	16	2	-32768 到+32767
unsigned short	16	2	0 到 65535
signed int	16	2	-32768 到+32767
unsigned int	16	2	0 到 65535
signed long	32	4	-2147483648 到+2147483647
unsigned long	32	4	0 到 4294967295
float	32	4	$\pm 1.175494\text{E}-38$ 到 $\pm 3.402823\text{E}+38$
bit	1		0 或 1
sbit	1		0 到 1
sfr	8	1	0 或 255
sfr16	16	2	0 到 65535

C51变量定义

C51变量定义的四要素:

【存储类别】

数据类型

【存储器类型】

变量名

(标准C)

(标准C+C51)

(C51特有)

(标准C)

unsigned char

data

sys_status

= 0;

* 括号项——可以缺省（但需有缺省值）


```
unsigned char    data    sys_status    = 0;
```

【存储类别】

数据类型

【存储器类型】

变量名

共有四个说明符：

- 1、**auto**（自动型）——变量的作用范围在定义它的函数体或语句块内。执行结束后，变量所占内存即被释放。 缺省存储种类为**auto (自动)型变量**
- 2、**extern**（外部型）——在一个源文件中被定义为外部型的变量，在其它源文件中需要通过**extern**说明方可使用。 声明变量或函数的外部链接属性，允许跨文件访问其他模块定义的全局变量或函数，常用于多文件编程中实现代码模块化。
- 3、**static**（静态型）——利用**static**可使变量定义所在的函数或语句块执行结束后，其分配的内存单元继续保留。
- 4、**register**（寄存器型）——将变量对应的储存单元指定为通用寄存器，以提高程序运行速度。 **C51**编译器编译时能自动识别程序中使用频率最高的变量，并自动将其作为寄存器变量，用户可以无需专门声明



```
unsigned char      data      sys_status      = 0;
```

【存储类别】 数据类型 【存储器类型】 变量名

数据的不同格式叫做数据类型

标准C语言的数据类型

数据类型		长度	值域
字符型 (char)	unsigned char	单字节	0~255
	signed char	单字节	-128~+127
整形 (int)	unsigned int	双字节	0~65535
	signed int	双字节	-32768~+32767
长整型 (long)	unsigned long	4字节	0~4294967295
	signed long	4字节	-2147483648~+2147483647
浮点型 (float)	float	4字节	10 ⁻³⁸ ~10 ³⁸
	double	8字节	10 ⁻³⁰⁸ ~10 ³⁰⁸
指针型	普通指针*	1-3字节	0~65535

* 有符号数类型可以忽略signed标识符

C51扩充数据类型: bit、sbit、sfr或sfr16



```
unsigned char      data      sys_status      = 0;
```

【存储类别】 数据类型 【存储器类型】 变量名

51单片机的 三个逻辑存储空间：
片内数据存储器，片外数据存储器 and 程序存储器。

类型 关键字	存储区	描述
data	DATA	单片机内部RAM空间的低128字节（00H-7FH），可在一个周期内直接寻址。
bdata	BDATA	DATA区中可以字节、位混合寻址的16字节（20H-2FH）位寻址区。
idata	IDATA	内部RAM 256字节（00H-FFH），必须采用间接寻址。
xdata	XDATA	片外RAM，地址范围64KB，使用DPTR间接寻址。
pdata	PDATA	片外RAM的低256字节(00H-FFH)，可通过P0口的地址对其寻址。
code	CODE	程序存储区64KB (0000H-FFFFH)，内容只读，使用DPTR寻址。

```
unsigned char    data    sys_status    = 0;
```

【存储类别】 数据类型 【存储器类型】 变量名

变量名可以由字母、数字和下划线三种字符组成，且第一个字符必须为字母或下划线，变量名长度随编译系统而定。

变量名具有字母大小写的敏感性，如SUM和sum代表不同的变量。

变量名不得使用标准C语言和C51语言的关键字。

```
unsigned char idata status_byte;
```

//定义status_byte为无符号字符型自动变量，该变量位于idata区

```
unsigned int code unit_id[2]={0x1234, 0x89ab};
```

//定义unit_id[2]为无符号整型自动变量，该变量位于code区中，是长度为2的数组，且初值为0x1234和0x89ab。

```
static char m, n;
```

//定义m和n为2个位于data区中的有符号字符型静态变量。

【例】变量定义存储种类和存储器类型相关情况。

```
char data var1;    /*在片内RAM低128B定义用直接寻址方式访问的字符型变量var1*/
```

```
int idata var2;    /*在片内RAM256B定义用间接寻址方式访问的整型变量var2*/
```

```
auto unsigned long data var3;    /*在片内RAM128B定义用直接寻址方式访问的自  
                                     动无符号长整型变量var3*/
```

```
extern float xdata var4;    /*在片外RAM64KB空间定义用间接寻址方式访问的外部  
                               实型变量var4*/
```

```
int code var5;    /*在ROM空间定义整型变量var5*/
```

```
unsign char bdata var6;    /*在片内RAM位寻址区20H~2FH单元定义可字节处理和位  
                               处理的无符号字符型变量var6*/
```

为使编程时书写简化，数据类型允许用缩写

例如：#define **uchar** unsigned char // **uchar** 作为 unsigned char的缩写

#define **uint** unsigned int // **uint** 作为 unsigned int的缩写

#define **ulong** unsigned long // **ulong** 作为 unsigned long 的缩写

#define LED P1 // 定义P1作为 (8个)LED的控制端口

#define initVal 0xfc // 定义initVal 值为 0xfc

则：

ulong xdata array[100]; //等价 unsigned long xdata array[100];

uint pdata dimension; // 等价 unsigned int pdata dimension;



4.2 C51的运算符及表达式

运算符和表达式方面，C51语言与标准C语言的核心的算术、逻辑关系和位运算符在语法和基本行为上是完全一致的。

基本算术运算符

4.2.1 算术运算符和算术表达式

右表运算符中，加、减和乘法符合一般的算术运算规则。

除法运算时，如果是两个整数相除，其结果为整数；如果是两个浮点数相除，其结果为浮点数。而对于取余运算，则要求两个运算对象均为整型数据。

符号	说明
+	加法运算
-	减法运算
*	乘法运算
/	除法运算
%	取模运算

% 模（求余）运算符；例11%3=2，结果是11除以3所得余数为2。

4.2.2 关系运算符与表达式

关系表达式是由关系运算符连接表达式构成的。

1. 关系运算符

符号	说明
>	大于
<	小于
>=	大于或等于
<=	小于或等于
==	等于
!=	不等于

表中上面的4种优先级高于后面的两种。

2. 关系表达式

由关系运算符组成的表达式，称为关系表达式。关系运算符两边的运算对象，可以是C语言中任意合法的表达式，例如：

`x>y;`

`(x=5) <= y;` //先赋值给x，再判断大小关系

关系表达式的值是整数0或1，其中0代表逻辑假；1代表逻辑真。在C语言中不存在专门的“逻辑值”。

逻辑运算符的结合性为从左到右。

优先级：算术运算符 > 关系运算符 > 赋值运算符

1) 关系运算符的低于算术运算符

如： $c > a + b$ // 等效于 $c > (a + b)$;

$a > b \neq c$ // 等效于 $(a > b) \neq c$

2) 关系运算符的优先级高于赋值运算符

如： $a = b > c$ // 等效于 $a = (b > c)$

$(x = 5) \leq y$; // 先赋值给x, 再判断大小关系, 则要用()

$x = 5 \leq y$; // 等效于 $a = (5 \leq y)$

3) 关系运算符的结合性为左结合

如： `int a=4, b=3, c=1;`

$f = a > b > c$ // $a > b$ 的值为1, $1 > c$ 的值为0, 故 $f=0$

4.2.3 逻辑运算符与表达式

1. 逻辑运算符

符号	说明
!	逻辑非(NOT)
&&	逻辑与(and)
	逻辑或(OR)

2. 逻辑表达式

由逻辑运算符组成的表达式称为逻辑表达式。逻辑运算符两边的运算对象可以是C语言中任意合法的表达式。

逻辑表达式的结果为1（结果为“真”时）或0（结果为“假”时）。

优先级：！（逻辑非）最高，&&（逻辑与）次之，||（逻辑或）低。

逻辑运算符具有自左至右的结合性。

逻辑运算符、赋值运算符、算术运算符、关系运算符之间优先级的次序：

优先级： **!** (逻辑非) > **算术运算符** > **关系运算符** > **&&** (逻辑与) > **||** (逻辑或) > **赋值运算符**

【例】： int a=4, b=5;

!a	为假, 因为a=4 (非0) 为真, 所以!a为假 (0) 。
a b	为真。因为a,b为真, 所以两者相或为真。
a && b	为真
!a && b	为假(0), 因为 !优先级高于&&, 因此先执行!a (结果为假,0) ⇒ 进而执行 0 && b=0 (结果为假, 0)。
(x=5) <= y	// 先给变量x赋值, 再执行关系运算;
x=5 <= y	先 比较 5 <= y, 再根据结果赋值给变量x

4.2.4 赋值运算符与表达式

1. 赋值运算符 (“ = ”)

格式: **〈变量名〉 = 表达式**

如: $a=3$, 表示把 3 赋给 变量 a

说明:

- 1) 赋值运算符为双目运算符, 即 “=”两边的变量名和表达式均为操作数, 一般情况下变量与表达式的值类型应一致。
- 2) 运算符左边只能是变量名, 而不能是表达式。

【例】:

```
int x=3,y=4, a;  
a=x+1<=y-1;
```

/* 根据运算符优先级, 等价于 $a = ((x+1) \leq (y-1))$, 即先对 =号右边式子: $x+1 \leq y-1$ 运算, 再根据 结果给a赋值;
式子运算顺序: 先执行**算术**运算 $(x+1)$ 和 $(y-1)$, 然后进行**关系** $\leq$ 判断, 再根据 结果给a**赋值***/

! (逻辑非) > **算术运算符** > **关系运算符** > **&&** (逻辑与) > **||** (逻辑或) > **赋值运算符**

2. 复合赋值运算符

在“=”前面加上双目运算符，如“<<”、“>>”、“+”、“-”、“*”、“%”、“/”等即构成复合赋值运算符。

+=	-=	*=	/=	%=	
>>=	<<=	&=	=	^=	~=

表4-5

按优先级顺序结合运算，所有复合赋值运算符级别相同，且与赋值运算符同一优先级，都具有右结合性。

$b+=4$ 等价于 $b=b+4$

$a>>=4$ 等价于 $a=a>>4$

$a+=b$ 相当于 $a=a+b$

$a-=b$ 相当于 $a=a-b$

$a*=b$ 相当于 $a=a*b$

$a/=b$ 相当于 $a=a/b$

$a\%=b$ 相当于 $a=a\%b$

$a<<=3$ 相当于 $a=a<<3$

$a>>=2$ 相当于 $a=a>>2$

..... 等等。

例如：

$a+=b$ 等价于 $a=(a+b)$

$x*=a+b$ 等价于 $x=(x*(a+b))$

$a\&=b$ 等价于 $a=(a\&b)$

$a<<=4$ 等价于 $a=(a<<4)$

4.2.5 自增和自减运算符与表达式

符号	说明
<code>++</code>	变量值加1
<code>--</code>	变量值减1

格式: `i++` `i--`

功能: 先使用 `i` 的值, 然后, 变量 `i` 的值增加 (减少) 1, 即 $i=i\pm 1$ 。

例如:

```
int a=3, b;
```

```
b=a++;
```

执行后, 则 **b** 的值为 **3**, **a** 的值为 **4**。

格式: `++i` `--i`

功能: 变量 `i` 先增加 (减少) 1, 即 $i=i\pm 1$, 然后, 再使用 `i` 的值。

对比:

```
int a=3, b;
```

```
b=++a;
```

执行后, 则 **b** 的值为 **4**, **a** 的值为 **4**。

`++` 和 `--` 运算符只能用于变量, 不能用于常量和表达式。 如: `++(a+1)` 是错误的



4.2.6 C51位操作及其表达式

表4-6

符号	含义
~	按位取反。(单目运算符)
<<	左移操作。左边移出的位舍去，右边移入的位补0
>>	右移操作。右边移出的位舍去，左边移入的位对无符号数补0； 对有符号数补符号位
&	按位与
^	按位异或
	按位或

位运算符操作的对象只能是整型或字符型数据

【例】按位取反，求 ~ 15 的值。

```
unsigned char x=15;    /* x为 00001111 B */  
P0=x;                 /* P0= 00001111 B */  
x = ~ x;              /* x取反后结果为 11110000 B */  
P0 = x;               /* P0 = 11110000 B */
```

移位运算符(<< 和 >>)

移位运算符是将一个数的二进制位向左或向右移若干位。移位后，空白位补0，而溢出的位舍弃。

以左移运算符为例，一般书写格式为：

表达式1 << 表达式2

其中“表达式1”是被移对象，“表达式2”给出移动位数。

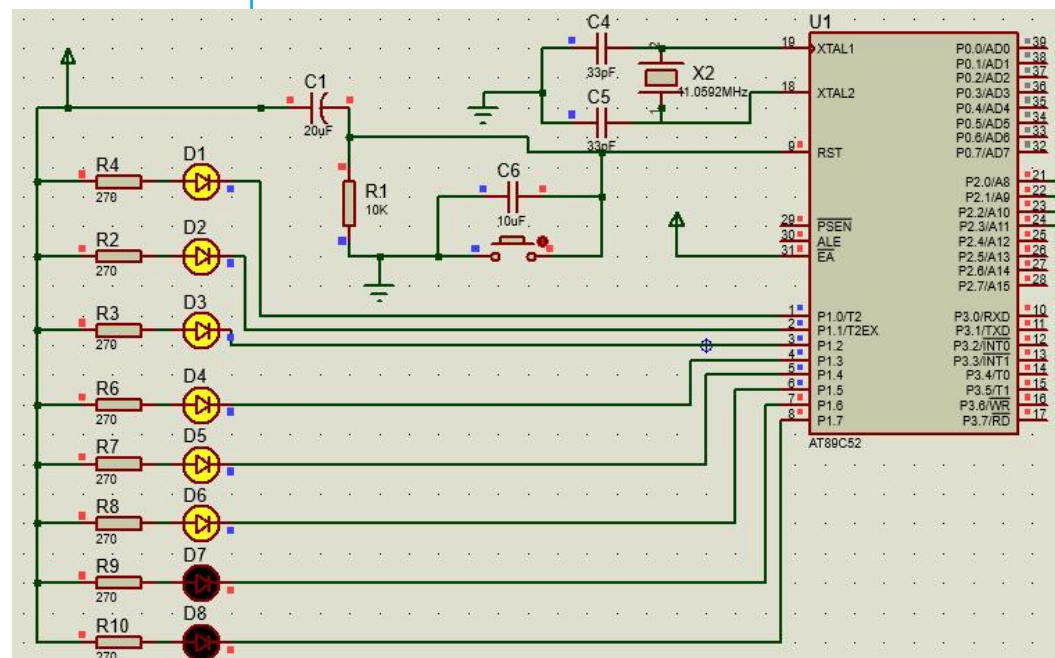
左移运算符是将其操作对象向左移动指定的位数，每左移1位相当于乘以2，移n位相当于乘以2的n次方。

【例】

```
unsigned char a = 0xcb;  
a = a<<2;  
/* 移位前 a = 1100 1011 B  
左移2位后 a = 0010 1100 B */  
a = a>>2;  
/* 移位前 a = 0010 1100 B  
右移3位后 a = 0000 1011 B */
```

【例4-1】对P1口输出控制8位LED发光二极管(共阳极), 左移循环每次点亮两支

```
#include <reg51.h>
#define LED P1           // 定义 LED的控制端口
#define initVal 0xfc     // 定义初始化值
#define uchar unsigned char
void delay(uchar m);     // 声明延时函数delay
void main()              // 主函数
{   LED=initVal ;        // 初始化 LED显示 1111 1110
    delay(20);
    while(1)
    {   LED = LED <<2 ; // 当前值按位左移LED两位, 然后赋值
        delay(20);
    }
}
```



```
void delay(uchar m)      //延时函数
{   uchar a,b,c;
    for(c=0;c<m;c++)
        for(a=0;a<100;a++)
            for(b=0;b<100;b++); //三层循环
}
```

移位运算符(<< 和 >>)移位后, 空白位补0, 而溢出的位舍弃。

移位运算符(<< 和 >>)移位后，空白位补0，而溢出的位舍弃。

如果需要**循环移位**，可以调用stdlib.h的函数，如下图的_crol_(char变量循环左移函数)，它将操作数的二进制位向左移动指定的位数，**与 << 不同的是，移出的高位会循环到低位端填入**，而不是舍弃。

crol(s_data , 2);

- _crol_,_cror_: 将char型变量循环向左(右)移动指定位数后返回
- _iror_,_irol_: 将int型变量循环向左(右)移动指定位数后返回
- _lrol_,_lror_: 将long型变量循环向左(右)移动指定位数后返回
- _nop_: 相当于插入NOP
- _testbit_: 相当于JBC bitvar测试该位变量并跳转同时清除。
- _chkfloat_: 测试并返回浮点数状态。 <intrins.h>

附录B

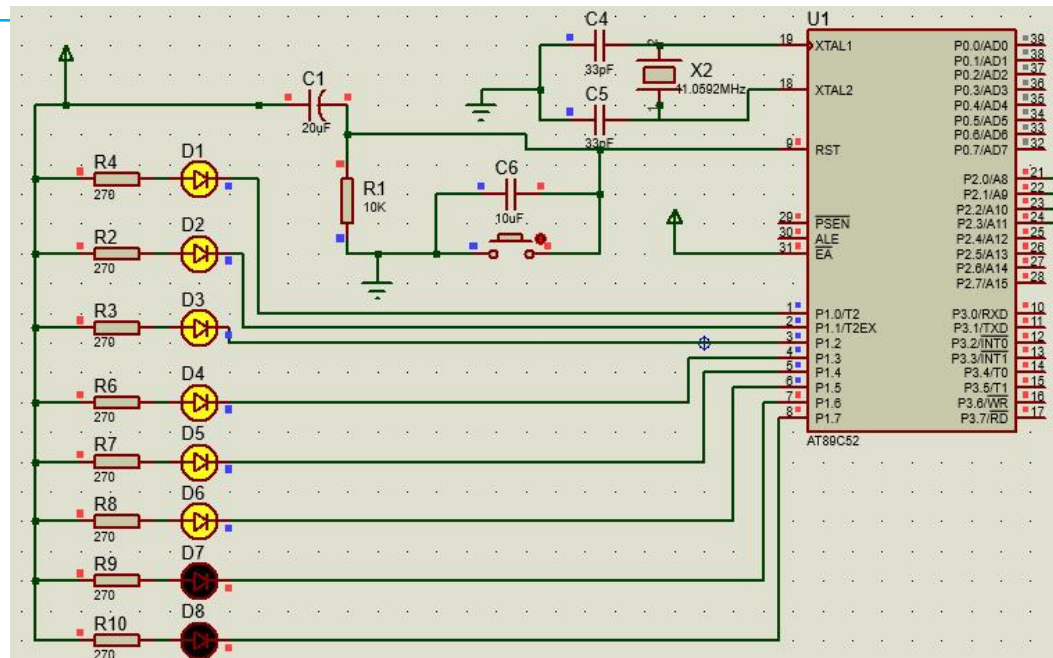
【例4-1_crol】对P1口输出控制8位LED发光二极管(共阳极), 左移循环每次点亮两支

```
#include <reg51.h>
#include <intrins.h>
#define uchar unsigned char
#define LED P1
#define initVal 0xfc

void delay(uchar m);    //声明延时函数delay

void main()
{
    uchar s_data = initVal;    // 点亮发光二极管的代码
    while(1)
    {
        LED =s_data;        // 更新LED点亮值
        s_data = _crol_(s_data , 2);    //s_data 按位循环左移两位
        delay(20);
    }
}
```

与 << 不同的是, **_crol_**移出的高位会循环到低位端填入



数据类型转换

运算符两侧的数据类型不同时，要转换成同种类型。转换的方法有两种：

(1) 自动转换，编译系统在编译时自动进行的类型转换，顺序是：

bit→char→int→long→float,signed→unsigned。

(2) 强制类型转换，通过类型转换运算来实现的。

一般形式：**(类型说明符) (表达式)**

功能：把表达式的运算结果强制转换成类型说明符所表示的类型。

【例】

```
long a, x, y, z=5.0 ;    // 初始化时是long类型
(double) a              // 将变量 a强制转换成double类型
(int)(x+y)               // 将x+y值强制转换成int类型
(float)(z %3)            //将模运算z%3的值强制转换成float类型
```

4.2.7 条件运算符与表达式

条件运算符格式：

表达式1 ? 表达式2 : 表达式3

执行过程：首先判断表达式1的值是否为真，如果是真，就将表达式2的值作为整个条件表达式的值，如果为假，将表达式3作为整个条件表达式的值。

例如：

```
max = (a>b) ? a:b
```

```
// 当a>b成立时, max=a;
```

```
// 当a<b不成立时, max=b;
```

等价于：

```
if(a>b) max =a;
```

```
else max = b;
```

4.3 C51控制语句

语句	功能
if...else...	条件语句
for()	循环语句
while()	循环语句
do...while()	循环语句
continue	结束本次循环语句
break	中止执行switch语句或循环语句
switch...case	多分支选择语句
goto	转向语句
return	从函数返回语句

4.3.1 条件语句

条件语句又称为分支语句，由关键字if构成，有以下三种基本形式：

1) 单分支条件语句：

if(条件表达式) 语句

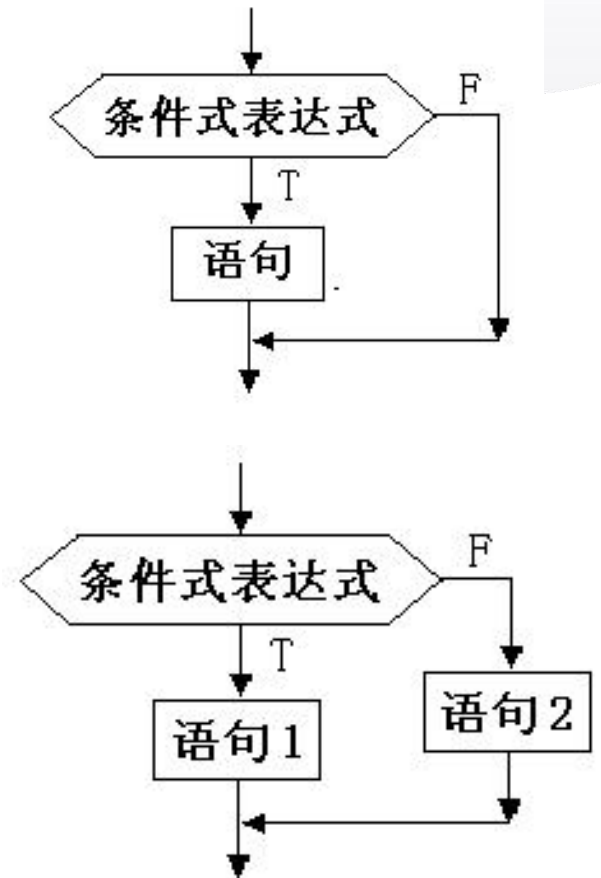
执行过程：如果括号里条件表达式结果为真，则执行括号后的语句。

2) 两分支条件语句：

if(条件表达式) 语句1

else 语句2

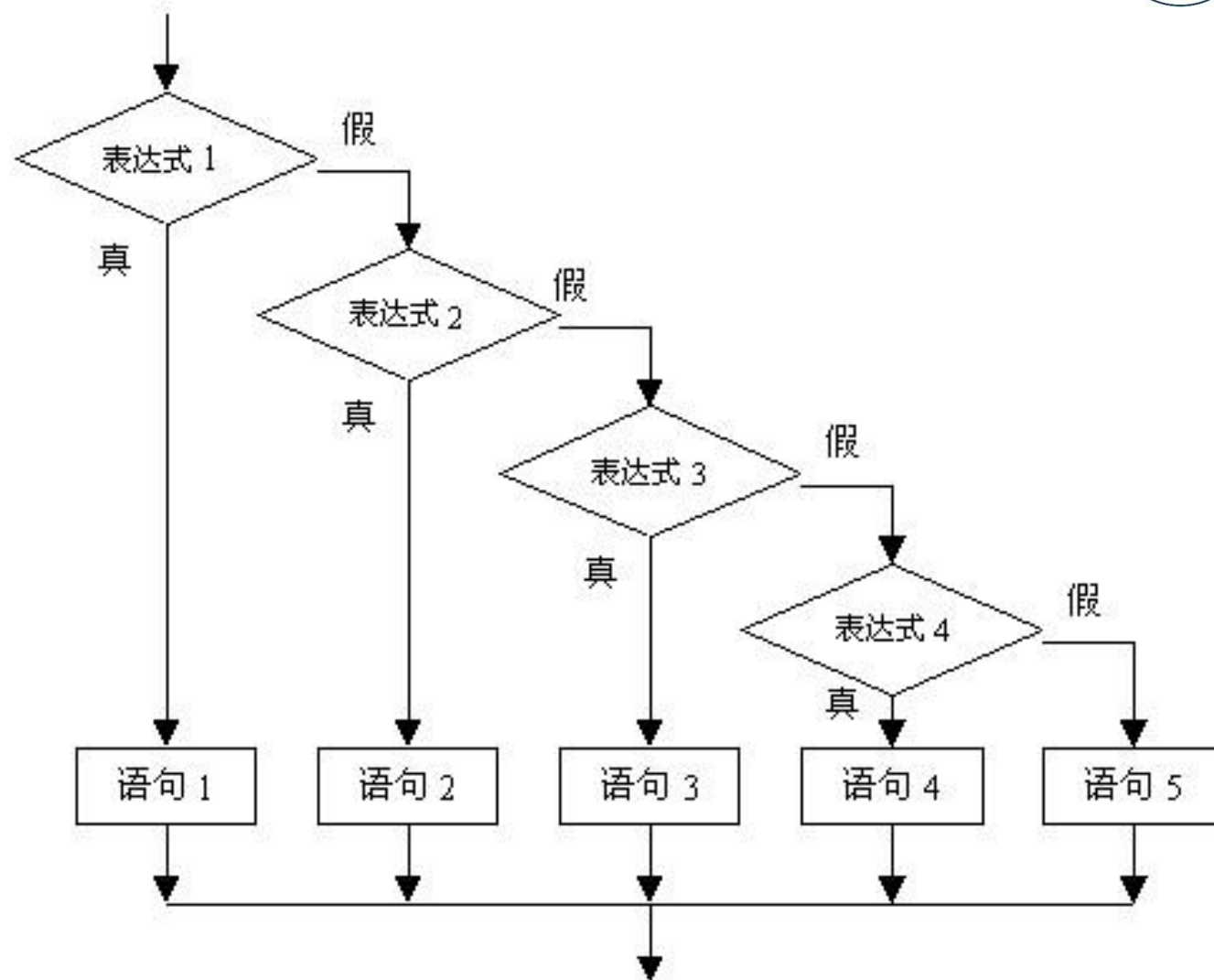
执行过程：如果括号里条件表达式结果为真，则执行语句1，否则（也就是括号里的表达式为假）执行语句2。



3) 多分支条件语句:

```
if(条件表达式1)语句1
  else if (条件表达式2) 语句2
    else if (条件表达式3) 语句3
      ⋮
    else if (条件表达式n) 语句m
      else 语句n
```

条件语句常用来实现多方向条件分支，其实，它是由if-else语句嵌套而成的，else总是与最邻近的if相配对。

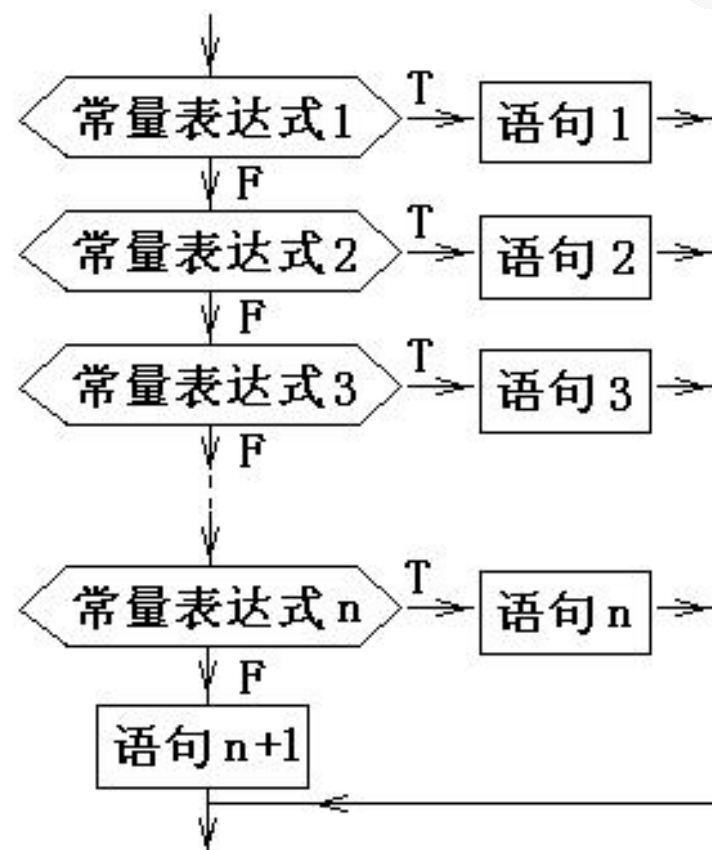


4.3.2 switch/case 语句

switch/case 语句专门用来处理多路分支的情形。

其格式如下：

```
switch (表达式)
{
    case 常量表达式1:
        {语句1;} break;
    case 常量表达式2:
        {语句2;} break;
    ⋮
    case 常量表达式n:
        {语句n;} break;
    default:
        {语句m;} break;
}
```



注意: case后的语句必须加break, 否则, 程序则顺移到下一个case继续执行。

□对switch语句需要注意:

- case分支中的常量表达式的值必须是整形、字符型或枚举类型，不能使用条件运算符。
- break语句用于跳出switch结构。若case分支中未使用break语句，则程序将继续执行下一个case分支中的语句直至遇到break语句或整个switch语句结束。这可以用于多个分支需要执行相同的语句进行处理的情况。

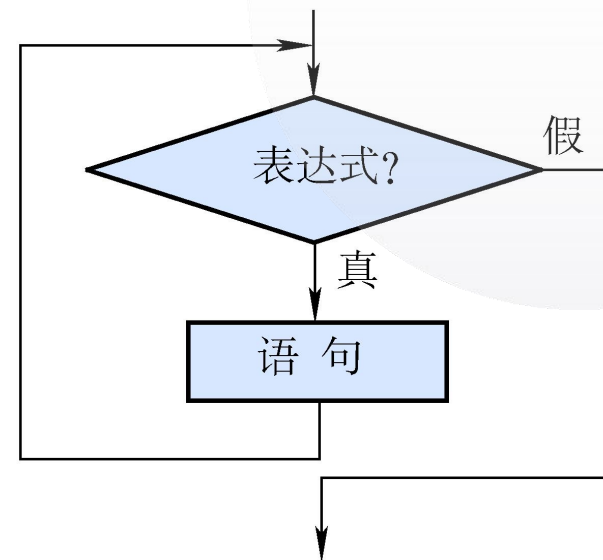
4.3.3 循环结构

1. while语句

一般形式如下：

while (条件表达式) 语句

执行过程：当条件表达式中的值为真，即非0时，执行后边的语句，如此循环，直到括号中的条件表达式为假时为止。



【例】 通过while语句 求1+2+3+...100的和。

```
int i=1, s=0;           //定义整型变量i和s
while (i<=100)
{
    s=s+i;              //累加1~100之和在s中
    i++;
}
```

2. do-while语句

一般形式如下：

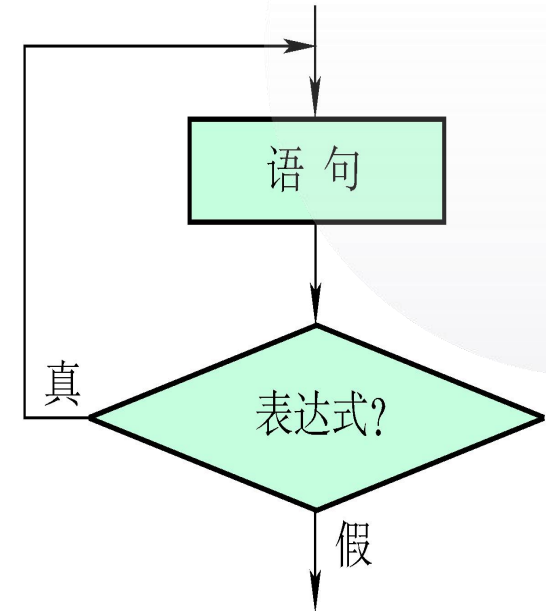
```
do  
    {语句;}  
while(条件表达式);
```

先执行给定的循环体语句，然后再检查条件表达式的结果。当条件表达式的值为真时，则重复执行循环体语句，直到条件表达式的值变为假时为止。

因此，用do-while语句构成的循环结构在任何条件下，循环体语句至少会被执行一次。

【例】 通过do-while语句 求1+2+3+...100的和。

```
int i=1, s=0;           //定义整型变量i和s  
do  
{  
    s=s+i;              //累加1~100之和在s中  
    i++;  
}while (i<=100)
```



3. for语句

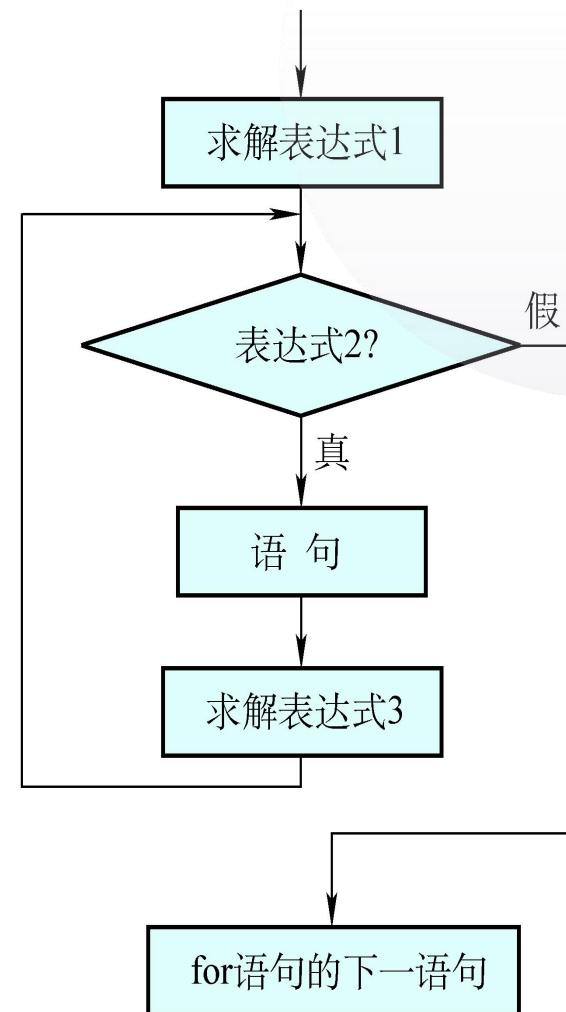
一般形式如下：

for ([表达式1]; [表达式2]; [表达式3])
语句

for语句的执行过程为：

- 1) 先求解表达式1，表达式1只执行一次，一般是赋值语句，用于初始化变量；
- 2) 求解表达式2，若为假（0），则结束循环；
- 3) 当表达式2为真（非0）时，执行循环体；
- 4) 执行表达式3；
- 5) 转回2) 重复执行。

使用时注意：表达式1，表达式2，表达式3和循环体均可以缺省。



【例】 求1+2+3+...100的和。

```
int i, sum=0;  
for(i=1; i<=100; i++) sum=sum+i;
```

4. 循环结构嵌套

一个循环体内包含另一个完整的循环结构，称为循环的嵌套。循环之中还可以套循环，称为多层循环。三种循环（while循环、do ... while循环和for循环）可以互相嵌套。

【例】通过循环嵌套程序实现延时

```
void msec(unsigned int x)
{ unsigned int i, j;
  while(x--)
  { for(i=0; i<125; i++)
    { for(j=0; j<80; j++)
      { ; }
    }
  }
}
```

/*外循环*/
/*嵌套内循环*/
/*嵌套内循环*/

```
void delay(uchar m)          //延时函数
{ uchar a,b,c;
  for(c=0;c<m;c++)
    for(a=0;a<100;a++)
      for(b=0;b<100;b++);    //三层循环
}
```

5. break和continue语句

break和continue语句通常用于循环结构中，用来跳出循环结构。但是二者又有所不同，下面分别介绍。

1) break语句

前面已介绍过用break语句可以跳出switch结构，使程序继续执行switch结构后面的一个语句。使用break语句还可以从循环体中跳出循环，提前结束循环而接着执行循环结构下面的语句。它不能用在除了循环语句和switch语句之外的任何其它语句中。

【例】 计算圆的面积，当计算到面积大于100时，由break语句跳出循环。

```
unsigned int r;  
float area;  
for (r=1; r<=10; r++) {  
    area=pi*r*r;  
    if (area>100) break;  
}
```

2) continue语句

continue语句用在循环结构中，用于结束本次循环，跳过循环体中continue下面尚未执行的语句，直接进行下一次是否执行循环的判定。

continue语句和break语句的区别在于：continue语句只是结束本次循环而不是终止整个循环；break语句则是结束循环，不再进行条件判断。

【例】 printf 输出100~200间不能被3整除的数，并统计其数量。

```
unsigned int i, j=0;
for (i=100; i<=200; i++)
{
    if (i%3==0) continue; //如果被整除，则跳出本轮循环
    printf("%d "; i);
    j++;
}
```

在程序中，当i能被3整除时，执行continue语句，结束本轮循环，跳过printf（）函数和统计，(转向下轮循环)，i 不能被3整除时才执行printf（）函数和统计。

【例4-4改】 P1口输出控制8位LED发光二极管(共阳极), K1按下时全点亮, 松开全熄灭

```
#include <reg51.h>
```

```
#define LED P1
```

```
sbit key1=P2^2; //图上的K3
```

```
void main()
```

```
{
```

```
    P2|=0x3c; //P2.2~2.5写入1,设置为输入模式
```

```
    while(1)
```

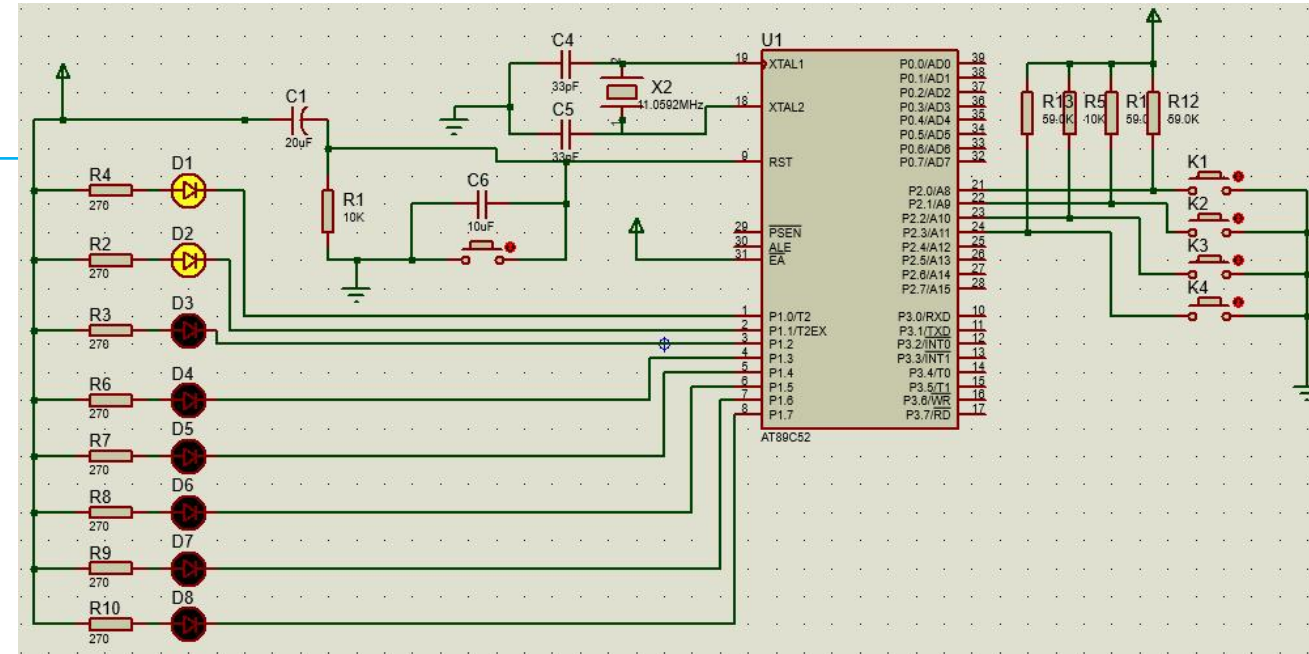
```
    {
```

```
        if(!key1) LED &=0xe1; //点亮P1.1~1.4 , 用1与操作, 不影响 P1的其它位    xxx0 000x
```

```
        else LED |=0x1e; //熄灭P1.1~1.4 , 用0或操作, 不影响 P1的其它位    xxx1 111x
```

```
    }
```

```
}
```



4.4 数组

数组是一种简单实用的数据结构，通过将多个变量（数据）人为地组成一定的结构，以便于处理大批量、相对有一定内在联系的数据。

4.4.1 一维数组的定义、引用、初始化

1. 一维数组

格式为：

类型标识符 数组名[常量表达式] , ... ;

例如：char ch[10]; //字符型数组，数组名为ch，含10个元素

int ina[6]; //整数型数组，数组名ina，含6个元素

说明：

- 1) **数组的下标从0开始**，分别是ch[0]、ch[1]、...、ch[9]。本例中不能使用ch[10]，因为超出正常范围了
- 2) 类型标识符 规定数组元素的数据类型，如本例中ch数组的每个元素都是char字符型数据；

2. 一维数组的引用

对于数值型数组，只能对数组元素逐个引用，如`ina[0]`，`ina[i]`，`ina[i+1]`等，不能引用整个数组。对于**字符数组**，可以逐个元素访问，也可以对整个数组按字符串的方式处理(`string.h`,附录B)。

3. 一维数组的初始化

C语言允许在定义数组时对各数组元素指定初始值——称为数组初始化

例如:

```
int idata a[5]={0, 1, 2, 3, 4} ;    /*声明片内RAM(256B)区的整型数组a,同时初始化
                                     数组元素, 括号{}内整型数据0,1,2,3,4分别赋给数组元素a[0], a[1], a[2],
                                     a[3], a[4] */
```

```
char ch[10];
unsigned int i;
for ( i=0; i<=9; i++){
    ch[i] = (char)i ; //转换为字符后保存到数组
}
```

注意:

- 1) 在定义数组时，若不对数组全部元素赋初值，则元素被缺省地赋值为0。
- 2) 只有在定义数组时才可以整体赋值，其他情况下整体赋值是错的。

4. 二维或多维数组

具有两个或两个以上下标的数组称为二维数组或多维数组，定义二维数组的一般形式为：

数据类型说明符 数组名[行数][列数]

- 数组名是一个标识符，行数和列数都是常量表达式；
- 二维数组可以在定义时进行整体初始化，也可以在定义以后单个进行赋值。

【例】

```
int a[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

或：

```
int a[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
```

5. 字符数组

若一个数组的元素是字符型的，则该数组就是一个字符数组。例如：

```
char a[10] = { "CHENG DU" };
```

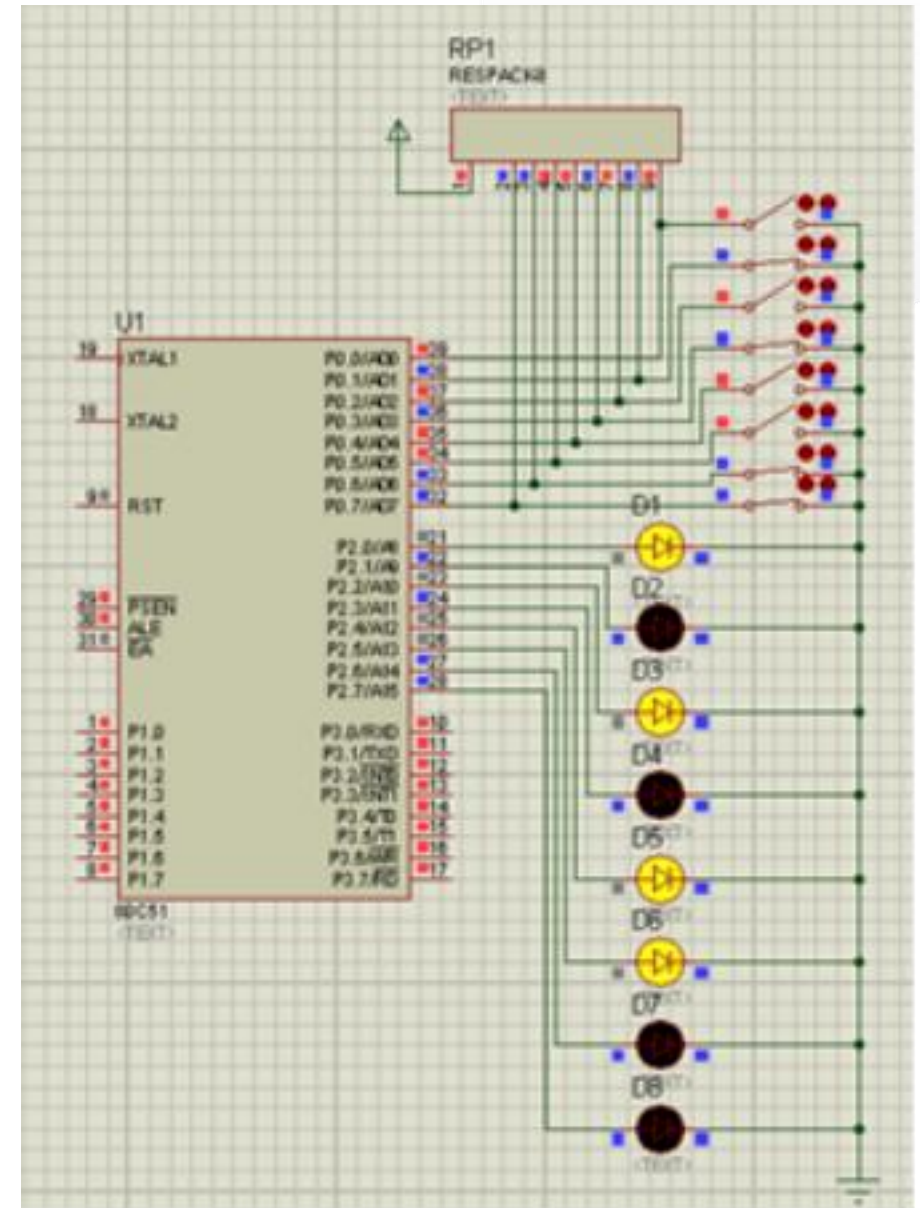
- 用**双引号**引起来的一串字符称为字符串常量，C51会自动在字符串末尾加上结束符' \0'。
- 用**单引号**括起来的字符表示 字符的**ASCII**码值，如' a'表示a的ASCII码值61H，而**双引号** " a"表示一个字符串，由两个字符(a和\0)组成。

```
#include <string.h>    //string.h 附录B
void main()
{
    char a[20] = { "CHENG DU" }; //部分初始化
    char b[20];
    strcat(a, " 12ab"); //a 后添加" 12ab": "CHENG DU 12ab"
    strcpy(b, a);    // 把a 字符串 拷贝 给 b
}
```

4.4.2 一维数组应用示例

【例】：用单片机实现将8位开关的输入状态通过8位LED（发光二极管）显示。

在Proteus ISIS下输入原理图，其中RESPACK-8为8位排阻作为P0口上拉电阻。首先单片机读入由P0口输入的8个开关量信息，则开关状态（闭合为低电平0、断开为高电平1）立即传送给P2口以控制8位LED显示器（二极管共阴极），当P2口某位为高电平时，则与其连接的发光二极管点亮。



C51控制程序

```
#include <reg51.h>
void delay(unsigned int);
void main() //主函数
{ unsigned char a[8];
  unsigned char i;
  P0= 0xff; //读入模式, 如果对P0没有其它操作, 放这
  while(1)
  { for(i=0; i<=7; i++)
    {
      a[i]=P2=P0; //读入开关状态, 并赋给P2和a数组
      delay(100); //调用延时子函数
    }
  }
}
```

E:\myProgExams\mcu_v4_\chapt4\c\exam_4_4_2.c

E:\myProgExams\myProteusExams 注意配hex路径

注意用英文的标点符号,
经常报错的是 :, ();

```
//延时子函数
void delay(unsigned int x)
{ unsigned char j;
  while(x--)
  {
    for (j=0; j<125;j++);
  }
}
```

4.5 函数

4.5.1 库函数及文件包含

- 1) C语言提供了丰富的标准函数，即**库函数**。这类函数是由系统提供并定义好的，不必用户再去编写。用户只需要了解函数的功能，并学会在程序中正确地调用库函数。
 - 2) 对每一类库函数，在调用该类库函数前，用户在源程序的include命令中应该包含该类库函数的头文件名（一般安排在程序的开始）。
 - 3) C51中的函数包括主函数(main)、库函数、**中断函数(interrupt)**、自定义函数及**可重入函数(reentrant)**。(表4-2)
- 文件包含通常还包括程序中使用的一些定义和声明。

C51常用的头文件:

```
#include <reg51.h>    /*定义51单片机内部资源在程序中的符号表示*/
#include <reg52.h>    /*定义52单片机内部资源在程序中的符号表示*/
#include <intrins.h>  /*调用本征函数(如移位函数)需要包含的头文件*/
#include <absacc.h>   /* 该文件中定义宏，以确定各存储空间的绝对地址。*/
#include <stdlib.h>   /* 动态内存分配函数*/
#include "math.h"     /*调用数学库函数前需要包含的头文件*/
#include <string.h>   /*调用字符串处理函数需要包含的头文件*/
#include "stdio.h"    /*调用C语言常用输入输出函数需要包含的头文件*/
```

见附录B

<intrins.h>

- `_crol_`, `_cror_`: 将char型变量循环向左(右)移动指定位数后返回
- `_iror_`, `_irol_`: 将int型变量循环向左(右)移动指定位数后返回
- `_lrol_`, `_lror_`: 将long型变量循环向左(右)移动指定位数后返回
- `_nop_`: 相当于插入NOP
- `_testbit_`: 相当于JBC bitvar测试该位变量并跳转同时清除。
- `_chkfloat_`: 测试并返回浮点数状态。

```
reg51.h
9  #ifndef __REG51_H__
10 #define __REG51_H__
11
12  /* BYTE Register */
13  sfr P0    = 0x80;
14  sfr P1    = 0x90;
15  sfr P2    = 0xA0;
16  sfr P3    = 0xB0;
17  sfr PSW   = 0xD0;
18  sfr ACC   = 0xE0;
19  sfr B     = 0xF0;
20  sfr SP    = 0x81;
21  sfr DPL   = 0x82;
22  sfr DPH   = 0x83;
23  sfr PCON  = 0x87;
24  sfr TCON  = 0x88;
25  sfr TMOD  = 0x89;
26  sfr TL0   = 0x8A;
```


4.5.2. C51自定义函数及调用

1. C51自定义函数

自定义函数语法格式如下：

```
返回值类型 函数名(形式参数表)[编译模式] [reentrant] [using n]
{
    函数体
}
```

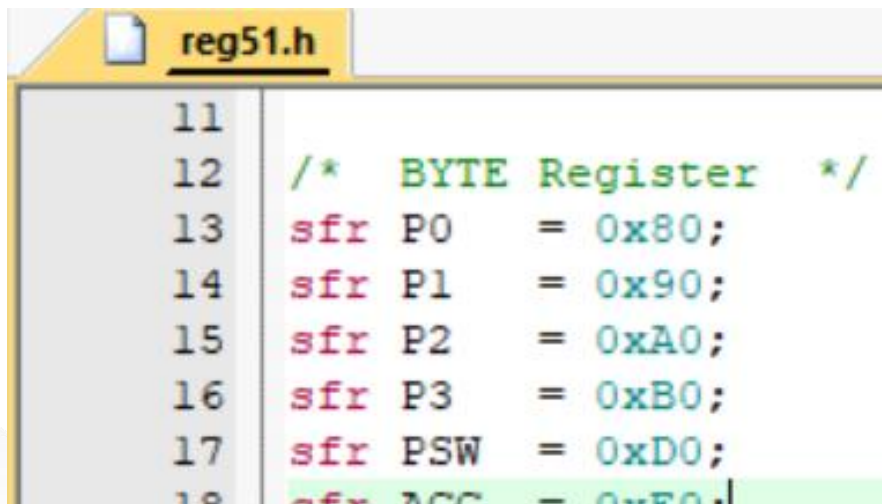
- 当函数无返回值时，应使用关键字void说明；
- 形式参数要分别说明类型，对于无形式参数的函数，则可在括号内填入void。
- 编译模式指存储模式，缺省时默认为SMALL（单片机内部存储区）。
- reentrant（可缺省）：函数是否可重入。
- using（可缺省）：指定函数所使用的寄存器组，n取值0~3。

using n 修饰符一般不用于有返回值的函数，因为C51函数的返回值是放在寄存器中，如寄存器组改变了，返回值会出错。

函数一般调用格式为：

函数名【实际参数表】

对于有返回值的函数，函数调用必须在需要返回值的地方使用；对于无返回值的函数，应该直接调用。



```
#include <reg51.h>           // 头文件
void delay(unsigned int);    // 声明自定义函数
void main()                  // 主函数
{
    unsigned char a[8];
    unsigned char i;
    while(1)
    {
        P0= 0xff;
        for (i=0; i<=7; i++)
        {
            a[i]=P2=P0;
            delay(100);      // 调用函数
        }
    }
}

void delay(unsigned int x) // 定义函数
{
    unsigned char j;
    while (x--)
    {
        for (j=0; j<125; j++);
    }
}
```

2. 函数调用的方式

1) **函数语句调用**-- 指把被调函数作为一个独立的语句直接出现在主调函数中。

`max(a,b);` // 调用有参子函数`max(a,b)`

`aver1();` // 调用无参函数`aver1()`。语句方式调用，一般是无返回值的函数

2) **在函数表达式中调用**

被调函数出现在主调函数中的表达式中，这种表达式称为函数表达式。

`c=5*max(a,b);` //先调用子函数`max(a,b)`，5*返回值，再赋值给c

3) **作为函数参数的调用**

被调函数作为另一个函数的参数时的调用，而另一个函数则是被调函数的主调函数。

`main()` // 主函数

{

`sum_mx(c, max(a,b));` // `sum_mx` 也是一个子函数

}

```
#include <reg51.h>           // 头文件
int sum(int data b[ ], int n); // 声明自定义函数
void main( ) //主函数
{   int a[8]={10, 20, 30, 40, 50, 90, 80, 70};
    int total;
    total= sum(a, 8); //调用自定义函数
}

int sum(int data b[ ], int n) // 函数定义，有返回值
{   int i, s=0;
    for(i=0; i<n; i++)
        s+=b[i];
    return s;           //返回值
}
```

函数调用时的实际参数必须与函数的形式参数在数据类型、个数及顺序完全一致。

3. 中断函数

中断函数定义语法格式如下：

```
void 函数名(void) interrupt n [using m]
{
    函数体
}
```

using m (可选项) : m 使用的寄存器组号

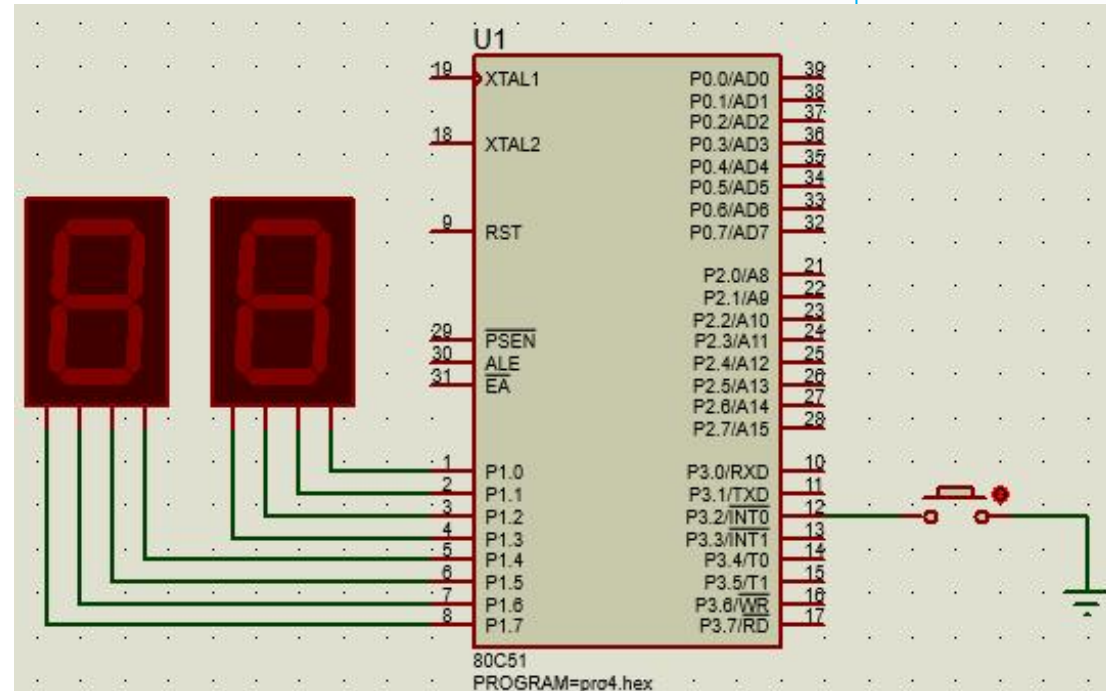
关键字 **interrupt** 定义该函数为中断服务函数, **n** 为中断号:

中断号0	→ 外部中断0	→ 0x0003	(地址)
中断号1	→ 定时/计数器0	→ 0x000b	(地址)
中断号2	→ 外部中断1	→ 0x0013	(地址)
中断号3	→ 定时/计数器1	→ 0x001b	(地址)
中断号4	→ 串口中断	→ 0x0023	(地址)

【例】编程实现中断函数对P3.2引脚按钮开关次数的统计。(例5-1)

```
#include<reg51.h>
unsigned int COUNT=0; /*定义全局变量COUNT*/

void main( )
{
    IE=0x81;          /*启用cpu和外部0中断*/
    TCON=0x01;        /* INT0设置为负边沿触发*/
    while (1){
        P1=COUNT;    /*输出计数结果*/
    }
}
```



```
void ex_int0(void)interrupt 0          /*定义外部中断0的中断函数*/
{
    COUNT++;      /*完成计数功能*/
}
```

4.6 指针

4.6.1 指针和指针变量

指针就是地址，是一种数据类型。

变量的指针就是变量的地址，存放地址的变量，就是指针变量。经C51编译后，变量的地址是不变的量。而指针变量可根据需要存放不同变量的地址，它的值是可以改变的。

1.定义指针变量

一般格式：

类型标识符 * 指针变量名

例如，

```
unsigned char *ch1, *ch2;    /*定义无符号字符型指针变量ch1,ch2*/
```

2. 指针变量的赋值

例如：

```
int i=2, a[5]={3,4,1,3,5}, *p1, *p2;    // 定义变量、数组和指针
p1= &i ;                               // 把变量 i 的地址 赋给 p1
p2= &a;                                 // 把数组 a 的首地址 付给 p2
```

指针的数据类型(比如本例p1, p2是int型), 应与指针所指向的地址所存放数据的类型一致!

3. 通用指针 与 存储区域指针

通用指针 指在定义指针变量时未说明其所在的存储空间。通用指针可以访问8051存储空间中与位置无关的任何变量。**int *p1, *p2; // p1,p2属于通用指针**

存储区域指针 指在定义指针变量的同时说明其存储器类型。

比如：

```
int data *q1;    // q1 片内存储器(0x00~0x7f)指针
char xdata *q2;  // q2 片外存储器指针
```


【例】下列程序定义了字符型存储区域指针，并使其指向相应存储区域的数组。

```
void main (void)
{
    char data *pd_c;  //定义指向(内部RAM)的字符变量指针变量pd_c
    char xdata *px_c; // 定义指向(外部RAM)的字符变量指针变量px_c
    char *p_c;        //通用指针
    char data a[10];  // 定义数组a，存储在内部RAM
    char xdata b[10]; // 定义数组b，存储在外部RAM
    pd_c = &a[0];      // 数组a的首地址赋给 指针pd_c
    px_c = &b[0];      // 数组b的首地址赋给 指针pd_c
}
```

类型 关键字	存储区	描述
data	DATA	内部RAM空间的低128字节（0x00-0x7f）
bdata	BDATA	DATA区中可以字节、位混合寻址的16字节（0x20-0x2f）位寻址区。
idata	IDATA	内部RAM 256字节（0x00-0xff）
xdata	XDATA	片外RAM，地址范围64KB，使用DPTR间接寻址。
pdata	PDATA	片外RAM的256字节（00H-FFH），可通过P0口的地址对其寻址。
code	CODE	程序存储区64KB（0x0000-0xffff），内容只读，使用DPTR寻址。

存储区指针的数据类型(比如本例px_c是char数据类型)，应与指针所指向的地址所存放数据的类型一致! (px_c指向的数组b存放的数据是char 型)。

可以不限定指针(可指向)的存储区，比如char *p_c 指针可以用于指向(片内RAM的)数组a和片外RAM的数组b

3. 指针变量的引用

可以通过指针运算符 “*” 引用指针变量。必须 **先定义指针变量**，**才能通过 * 引用**。

【例4-8】：

```
void main(void )
{
    int a , b ;           // int 型变量
    int *p1,*p2 ;         // 定义(int型)指针变量p1、 p2
    a=10 , b=20 ;
    p1=&a, p2=&b ;        // (int型)变量a、 b的地址分别赋给p1、 p2
    (*p1)++, (*p2)++ ;    // p1、 p2指向的变量(a和b)的数据自增1 , *p1 *p2 是引用
}
```

对比： ***p1++** 执行时，p1 所指向的地址变量先自增1，再引用该地址的数据。

而： **(*p1)++** 是将p1 所指向地址的数据自增1，但p1 所指向的地址未变

4.6.3 一维数组与指针

一维数组中，数组名可以表示第1个元素的地址，即该数组的起始地址。因此，可以用数组名方式，通过指向运算符 “*” 引用数组元素。

指针变量是存放地址的变量，也可以将指针变量指向一维数组，通过指针变量引用数组元素。

例如：

```
int a[10], m, *p, *q ; // 定义a数组和指针变量p
p=a ;                // 数组a的首地址赋给 p , 也可以 p = &a
q=&m;                 // 变量m的地址赋给 q
```

【例4-9】用不同的方法将数组a中的元素赋给b数组。

```
void main()
{   int a[10]={0,1,2,3,4,5,6,7,8,9},b[10],*p,i ;
    p=a ;
    for (i=0 ;i<=9 ;i++)
        b[i]=a[i];      /*通过a[i]直接引用数组元素*/
    for (i=0 ; i<=9 ;i++)
        b[i]=*(a+i);    /*通过* (a+i) 数组指针引用数组元素*/
    for (i=0 ; i<=9 ;i++)
        b[i]= *(p+i);   /*通过* (p+i) 数组指针引用数组元素*/
    for (i=0 ; i<=9 ;i++)
        b[i]= p[i];     /*通过p[i] 数组指针引用数组元素， 以上4条语句是等价的*/
    for (i=0; i<=9;i++) // 缺i++的话，i一直为0，无限次执行，且只更新b[0]
        b[i]=* p++;     /*通过*p引用数组元素，移动指针 (p++) */
    }                  /* 指针变量P递增*/
```

对比： *p1++ 执行时，p1 所指向的地址变量先自增1，再引用该地址的数据。

而： (*p1)++ 是将p1 所指向地址的数据自增1，但p1 所指向的地址未变

4.6.4 指向数组的指针作为函数参数

【例4-10】数组a的10个数据，调用函数(选择法排序)实现数组数据排序(从大到小)。

```
#include <reg52.h>
#define uint unsigned int
void selection_sort(uint arr[], uint n);

void main(){
    uint a[10],b[10]={38,16,20,79,17,28,93,20,41,56},*p=a,i;

    for(i=0;i<10;i++)
        *p++ = b[i];    // 先p++指向下一地址(p指向的地址值改变了), 再把b[i]赋给 p指向的地址
    p=a;    //恢复指向数组a的首地址
    selection_sort(p,10); //通过指针p将数组a的首地址赋给函数
    i=0;    // 加个语句, 只是为了调试时方便查看排序(函数执行)返回后的结果(数组a被排序了)
}
```

```

/*选择排序函数： 参数: arr - 待排序的数组指针,
   n - 数组元素个数*/
void selection_sort(uint arr[], uint n) {
    uint i, j, largerIdx, temp;
    // 遍历数组的每个位置
    for (i = 0; i < n - 1; i++) {
        largerIdx = i; // 假设当前位置i的元素是大值

        // 在未排序部分寻找大值
        for (j = i + 1; j < n; j++) {
            // 如果找到更大的元素, 更新大值索引
            if (arr[j] > arr[largerIdx]) {
                //如果换成< 则 从小到大 排序
                largerIdx = j;
            }
        }
    }
}

```

```

        // 如果大值不在当前位置, 则交换元
        素
        if (largerIdx != i) {
            temp = arr[i];
            arr[i] = arr[largerIdx];
            arr[largerIdx] = temp;
        }
    }
}

```

4.7 Keil 51单片机集成开发环境使用

1. 创建工程

选择Keil开发环境Project菜单，选择New Project菜单项来建立一个新的工程。

2. 选择目标芯片

在工程建立完成后，器件选择窗口便会弹出，从器件库选择单片机应用系统所使用的8051芯片。

3. 创建并编写C51源文件

选择C51开发环境的File菜单的New菜单项，这时会出现新建源程序编辑窗口，新建一个C语言源文件。

4. 把源程序文件加入到工程中

在建立了源程序文件后，必须把源程序文件添加到工程中，以构成一个完整的工程项目。

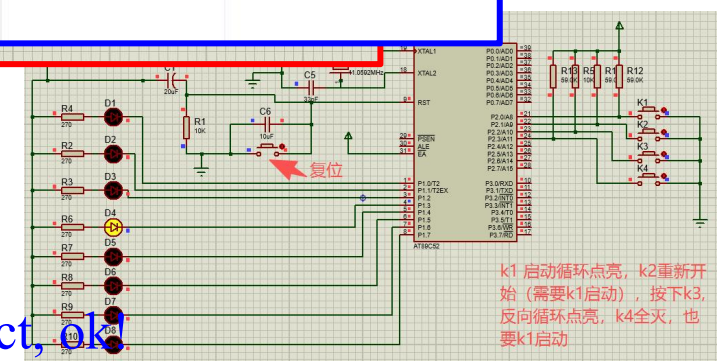
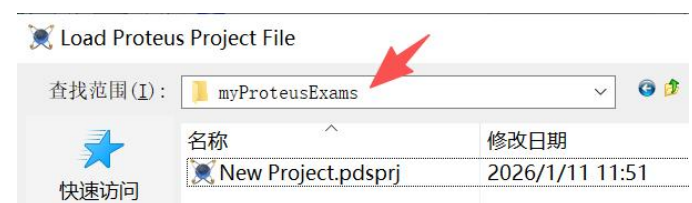
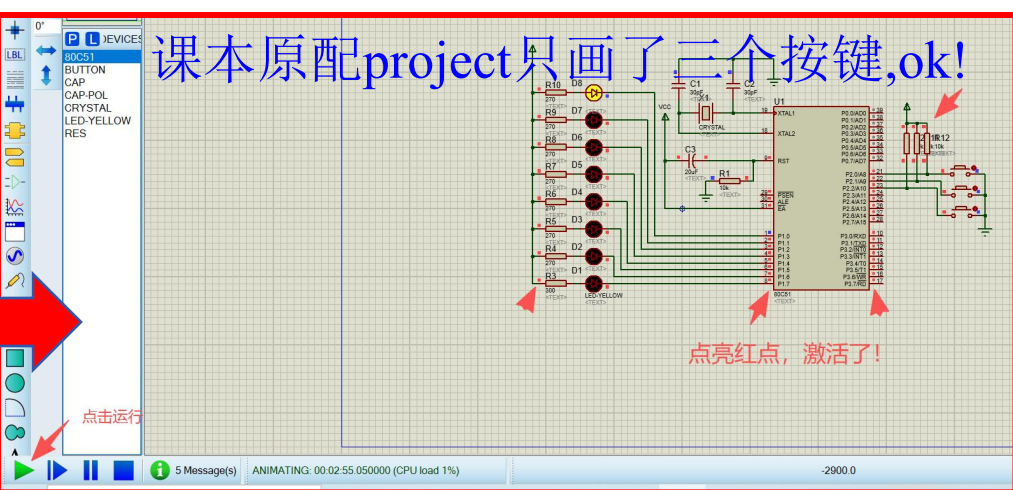
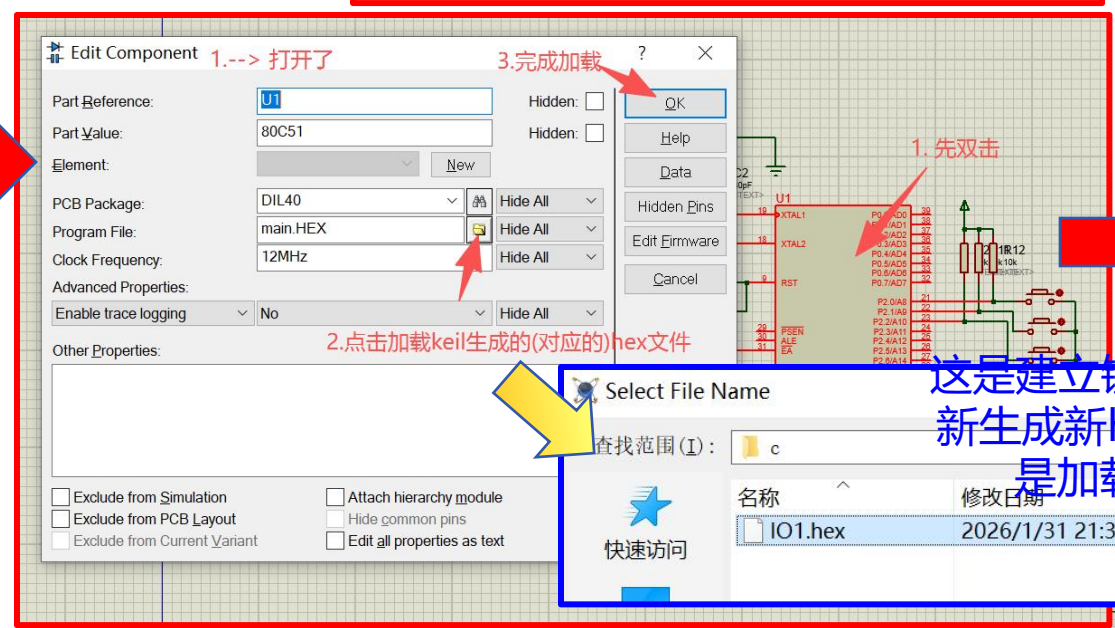
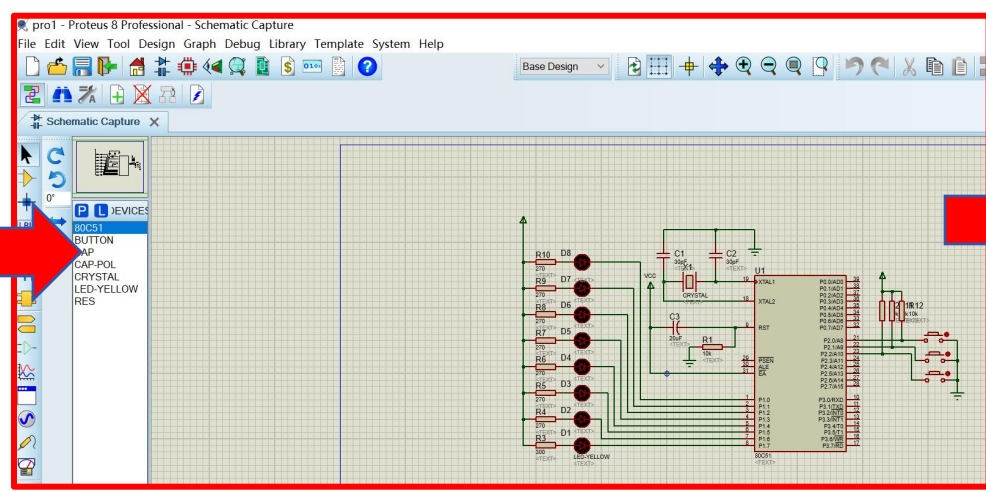
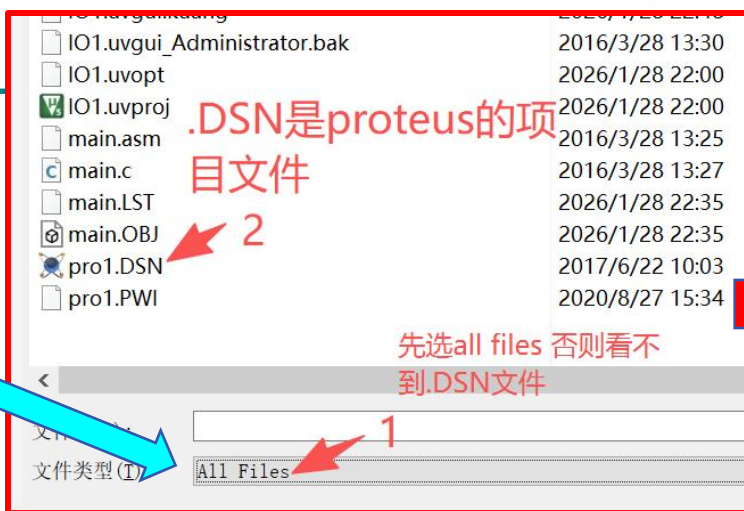
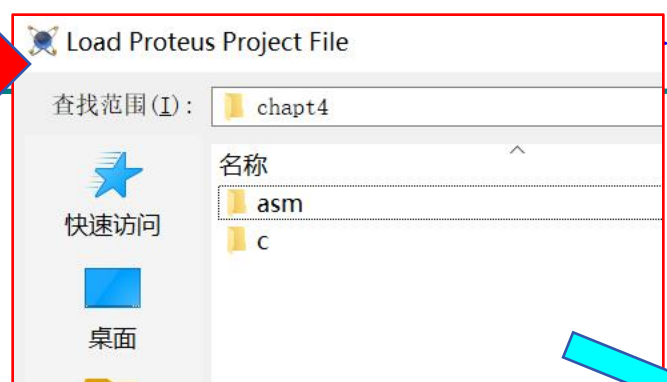
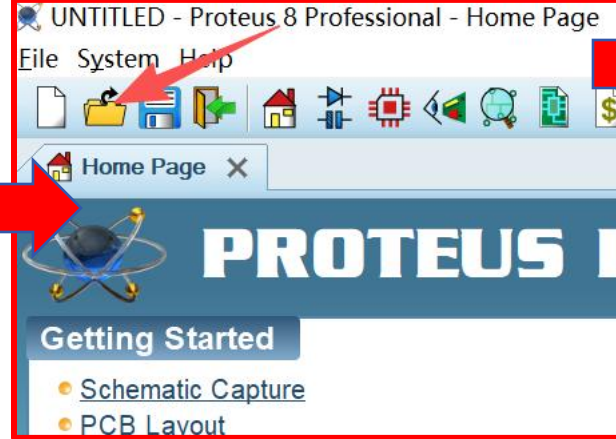
5. 编译生成HEX文件

选择C51开发环境的Project菜单中的Build target菜单项，则对工程中的文件进行编译、汇编和连接，生成.Hex文件。

6. 调试程序

对程序进行仿真功能验证及调试。Keil内置的软件仿真模块，可实现对51单片机的内部资源及IO口进行简单的仿真调试。

在调试状态下，目标文件可自动转换为HEX文件。在调试中可以对单片机的寄存器、内存、所产生的汇编代码、串口的输出情况等进行监视。



这是建立链接, 如果keil重新生成新hex, proteus也是加载新hex文件

按课本图, 新作的四个按键project, ok!

k1 启动循环点亮, k2重新开始 (需要k1启动), 按下k3, 反向循环点亮, k4全灭, 也要k1启动

● T H A N K Y O U

THANKS

C语言规定了算术运算符的**优先级和结合性**。

优先级---指当运算对象两侧都有运算符时，执行运算的先后次序。按运算符优先级别的高低顺序执行运算。

结合性---指当一个运算对象两侧的运算符优先级别相同时的运算顺序。

算术运算符中取负运算的优先级最高，其次是乘法、除法和取余，加法和减法的优先级最低。也可以根据需要，在算术表达式采用括号来改变优先级的顺序。

如：

$a+b/c$; /* 该表达式中，除号优先级高于加号，故先运算 b/c 所得结果，之后再与 a 相加。*/

$(a+b) * (c-d) - e$; /*该表达式中，括号优先级最高，其次是“*”，最后是减号。故先运算 $(a+b)$ 和 $(c-d)$ ，然后再将二者结果相乘，最后与 e 相减。*/

C51运算符的优先级和结合性

优先级		运算符	结合性
最高 ↑		() [] -> .	自左向右
		! ~ ++ -- + - * & sizeof	自右向左
		* / %	自左向右
		+ -	自左向右
		<< >>	自左向右
		< <= > >=	自左向右
		== !=	自左向右
		&	自左向右
		^	自左向右
			自左向右
		&&	自左向右
			自左向右
		?:	自右向左
		= += -= *= /= %= &= ^= = <<= >>=	自右向左
最低		,	自左向右